

MAXIME BRONNY

19009314

- BD1 - CADRE LOGICIEL POUR LE BIG-DATA

- RAPPORT DE PROJET HADOOP -

ANALYSE BIG DATA DE LA CONSOMMATION ÉLECTRIQUE (HDFS / YARN / MAPREDUCE / HIVE)

PRÉSENTATION

PowerConsumption-Hadoop-Analytics est un projet Hadoop réalisé dans le cadre de l'UE Cadre logiciel pour le Big Data. L'objectif est de mettre en œuvre une chaîne de traitement batch complète et reproductible pour analyser un dataset public de consommation électrique (household_power_consumption.txt, ~2,07M lignes / ~127 MB).

Les données sont ingérées dans HDFS, puis traitées par 3 jobs MapReduce (Java) exécutés sous YARN (agrégation journalière, détection de pics, comparaison semaine/week-end).

Les résultats sont ensuite structurés et interrogés via Hive (tables externes + requêtes analytiques), exportés en CSV, puis visualisés en Python (pandas/matplotlib/seaborn) pour produire les graphiques finaux. Le dépôt inclut les scripts d'exécution, les fichiers Hive (.hql), les exports et une démonstration vidéo du pipeline de bout en bout.

[HTTPS://GITHUB.COM/CRYZHIX/IED-
CADRELOGICIEL-POWERCONSUMPTION-
HADOOP-ANALYTICS](https://github.com/cryzhix/ied-cadrelogiciel-powerconsumption-hadoop-analytics)

UNIVERSITÉ
PARIS8
VINCENNES-SAINT-DENIS

Glossaire

1. Stockage & organisation (HDFS)

- **HDFS** : stockage des données brutes, des sorties MapReduce et des exports Hive.
 - **Chemin HDFS** : chemin côté cluster (ex. /user/projet/data/raw), distinct d'un chemin local Linux.
 - **/user/projet/** : racine HDFS du projet (données, outputs, exports).
 - **/user/projet/data/raw** : entrée du pipeline (dataset brut).
 - **/user/projet/output/** : sorties MapReduce (un répertoire par job).
 - **/user/projet/export/** : exports Hive (un répertoire par analyse).
 - **part-r-00000** : fichier de sortie d'un reducer (résultat final d'un job).
 - **_SUCCESS** : marqueur HDFS indiquant qu'un job s'est terminé sans erreur bloquante.
-

2. Exécution (YARN)

- **ResourceManager** : planifie les applications et alloue CPU/RAM.
 - **NodeManager** : exécute les containers sur le nœud et remonte l'état au ResourceManager.
 - **Container** : environnement d'exécution YARN (ressources allouées + commande).
 - **Application ID** : identifiant YARN d'un job (utile pour retrouver statut et logs).
 - **UI YARN (8088)** : interface web de suivi des jobs MapReduce.
 - **HADOOP_CLASSPATH** : classpath Hadoop utilisé pour exécuter correctement les drivers Java avec les libs Hadoop.
-

3. Hive (tables & exports)

- **Base consommation_elec** : base Hive qui regroupe les tables et requêtes du projet.
- **Table externe (EXTERNAL TABLE)** : table Hive qui pointe vers des fichiers déjà présents dans HDFS (pas de copie).
- **LOCATION** : chemin HDFS associé à une table externe.
- **FIELDS TERMINATED BY** : séparateur de colonnes pour les tables texte (ici ; sur les données brutes).
- **skip.header.line.count** : ignore la ligne d'en-tête lors de la lecture Hive.
- **serialization.null.format** : interprète ? comme NULL côté Hive.
- **INSERT OVERWRITE DIRECTORY** : export Hive dans un répertoire HDFS (résultat d'une requête).

- `\\N` : représentation d'une valeur NULL dans certains exports Hive.
 - `UNIX_TIMESTAMP(date,'dd/MM/yyyy')` : parsing des dates du dataset (format européen).
-

4. Scripts du projet (repères)

- `scripts/hdfs/create_structure.sh` : création des répertoires HDFS /user/projet/....
 - `scripts/execution/build_mapreduce.sh` : compilation Maven du module MapReduce (production du JAR).
 - `scripts/execution/run_mapreduce_jobs.sh` : exécution des 3 jobs MapReduce et contrôle rapide des sorties.
 - `hive/scripts/05_exports.hql` : exports Hive vers /user/projet/export/....
 - `scripts/export/export_results.sh` : rapatriement HDFS → local via `hdfs dfs -getmerge`.
 - `hdfs dfs -getmerge` : fusion de fichiers d'un répertoire HDFS en un seul fichier local.
 - `scripts/visualization/generate_graphs.py` : génération des PNG à partir des CSV exportés.
-

Table des Matières

PowerConsumption-Hadoop-Analytics	2
Glossaire	2
1. Stockage & organisation (HDFS)	2
2. Exécution (YARN)	2
3. Hive (tables & exports)	2
4. Scripts du projet (repères)	3
Table des Matières	4
1. Introduction & objectifs	7
1.1 Contexte et problématique	7
1.2 Objectifs techniques	7
1.3 Objectifs analytiques	7
1.4 Livrables	8
2. Recherche & planification	8
2.1 Démarche de recherche	8
2.2 Découpage du projet	8
2.3 Risques identifiés et mitigation	8
2.4 Planning	9
3. Données (dataset) & préparation	10
3.1 Source du dataset	10
3.2 Volume et caractéristiques	10
3.3 Format et schéma des données	10
3.4 Préparation et nettoyage	11
4. Architecture & déploiement Hadoop	12
4.1 Environnement de déploiement	12
4.2 Versions et dépendances logicielles	12
4.3 Architecture des services Hadoop	13
HDFS (stockage) :	13
YARN (orchestration / supervision) :	13
Hive (métadonnées + requêtes + exports) :	13
Interaction (flux simple) :	13
4.4 Configuration YARN (ressources)	14
4.5 Variables d'environnement et accès	15
4.6 Interfaces web et supervision	15
5. Ingestion HDFS & organisation des données	17
5.1 Organisation HDFS du projet	17

5.2 Création de la structure HDFS	18
5.3 Chargement des données	18
6. Développement MapReduce (sous YARN)	18
6.1 Build et packaging des jobs	18
6.2 Exécution des jobs MapReduce	19
6.3 Description des traitements MapReduce	19
6.4 Exécution sous YARN et suivi	20
6.5 Gestion des erreurs et validations	20
7. Data Warehouse Hive : schéma + requêtes analytiques	23
7.1 Création de la base de données	23
7.2 Modélisation des tables Hive	23
7.3 Requêtes analytiques	26
7.4 Interprétation synthétique des résultats	30
8. Exports & visualisations	31
8.1 Exports Hive vers HDFS	31
8.2 Rapatriement local des résultats	31
8.3 Génération des visualisations	32
8.4 Analyse des résultats et interprétation des visualisations	35
8.4.1 Analyse des pics et journées atypiques (Q1)	35
8.4.2 Évolution temporelle et saisonnalité (Q2, Q5)	36
8.4.3 Patterns journaliers et horaires (Q3, Q4)	37
8.4.4 Synthèse et enseignements	38
9. Tests, validations & qualité	39
9.1 Tests réalisés	39
9.2 Validations fonctionnelles	39
9.3 Indicateurs de qualité et limites	39
10. Conclusion, limites, perspectives	40
11.1 Bilan du projet	40
11.2 Limites identifiées	40
11.3 Perspectives d'amélioration	40
11. Bibliographie	41
1. Documentation technique officielle	41
Apache Hadoop	41
Apache Hive	41
LanguageManual	42
2. Dataset et sources de données	42
3. Guides et tutoriels techniques	42
Installation et configuration	42
Références Java MapReduce	42

1. Introduction & objectifs

1.1 Contexte et problématique

Ce projet analyse des mesures de consommation électrique domestique collectées à la minute (dataset Household Power Consumption).

Le besoin est de produire des indicateurs interprétables à plusieurs granularités temporelles (heure, jour, mois, année) pour caractériser les variations de consommation.

Le traitement visé est batch et doit être rejouable : l'objectif est de démontrer une chaîne Hadoop complète, reproductible et traçable.

Les données sont ingérées et organisées dans HDFS sous une arborescence projet (/user/projet/...) afin de servir de source unique aux traitements.

Les calculs sont exécutés en jobs MapReduce sous YARN, permettant le pilotage des ressources et l'audit de l'exécution (logs, état des applications).

Les résultats sont interrogés et exportés via Hive, puis restitués sous forme d'exports CSV et de visualisations PNG.

1.2 Objectifs techniques

- **HDFS** : Ingérer le dataset et standardiser les emplacements HDFS d'entrée/sortie (/user/projet/data/raw, /user/projet/output, /user/projet/export) (HDFS NameNode UI : <http://localhost:9870>) .
- **YARN** : Exécuter et suivre les applications de traitement (états, logs) via le ResourceManager (UI <http://localhost:8088>) .
- **MapReduce (Java)** : Implémenter 3 jobs batch (agrégation journalière, pics journaliers, comparaison semaine/week-end) et écrire les résultats dans HDFS.
- **Hive** : Créer la base "consommation_elec", définir les tables externes sur HDFS, exécuter les requêtes Q1-Q5 et produire les exports.(Hive metastore : <thrift://localhost:9083> & HiveServer2 : <jdbc:hive2://localhost:10000>)

1.3 Objectifs analytiques

- **Q1** : Repérer les jours atypiques en classant les journées par consommation moyenne.
- **Q2** : Mettre en évidence tendance et saisonnalité via une agrégation mensuelle.
- **Q3** : Quantifier l'effet "week-end" en comparant des statistiques de consommation entre semaine et week-end.
- **Q4** : Identifier les heures de pointe à partir de la distribution horaire de la consommation.
- **Q5** : Comparer les pics annuels afin de caractériser l'évolution des maxima dans le temps.

Ces analyses fournissent une lecture temporelle multi-échelle pour interpréter les variations de consommation, sans objectif de prédiction.

1.4 Livrables

Les livrables produits sont :

- Code MapReduce (Java) + JAR compilé
 - Scripts Hive (.hql)
 - Scripts Bash d'exécution/orchestration
 - Exports CSV
 - Visualisations PNG
 - Vidéo de démonstration
 - Rapport PDF
-

2. Recherche & planification

2.1 Démarche de recherche

Le sujet a été retenu pour répondre à un besoin simple et mesurable : analyser des variations de consommation électrique dans le temps.

Le dataset "UCI Household Electric Power Consumption" a été choisi car il est public, documenté et exploitable en batch (fichier texte ";", dates "DD/MM/YYYY", valeurs manquantes "?").

Son volume (127 MB, 2 075 260 lignes) est suffisant pour justifier un traitement Hadoop tout en restant compatible avec un projet individuel.

La temporalité (1 mesure/minute) permet de travailler sur plusieurs granularités (heure, jour, mois, année) sans enrichissement externe.

La démarche a consisté à construire un pipeline reproductible "HDFS → MapReduce/YARN → Hive → exports", puis à valider chaque étape par des commandes et sorties de scripts.

2.2 Découpage du projet

Le projet a été découpé en 5 phases, chacune reliée à une brique Hadoop ou à un livrable :

1. **Ingestion HDFS** : Création de l'arborescence "/user/projet/..." et dépôt du dataset dans "/user/projet/data/raw".
2. **Packaging MapReduce** : Compilation Maven et génération du JAR exécutable.
3. **Traitements MapReduce sous YARN** : exécution des 3 jobs batch et écriture des résultats dans "/user/projet/output/...".
4. **Analyse Hive** : Création de la base "consommation_elec", tables externes sur HDFS et exécution des requêtes Q1–Q5.
5. **Exports & restitution** : Exports Hive dans "/user/projet/export/...", exporté également en fichiers CSV locaux et génération des visualisations PNG.

2.3 Risques identifiés et mitigation

Risque	Impact	Mitigation
Services HDFS/YARN/Hive non démarrés ou configuration incohérente	Bloquant	Vérifications "fail-fast" dans les scripts ("hdfs dfsadmin -report", prérequis) + contrôle via UI YARN http://localhost:8088 .
Formats d'entrée hétérogènes (séparateur ";", en-tête, valeurs ?)	Moyen	Paramétrage Hive ("skip.header.line.count", "serialization.null.format") + filtrage/validation côté MapReduce.
Incompatibilité de formats MapReduce ↔ Hive/exports (TAB + valeurs concaténées)	Moyen	Documenter le format, adapter les tables externes et la chaîne export/visualisation ("hive/scripts/02_create_tables.hql" "scripts/visualization/generate_graphs.py").
Classpath/dépendances Hadoop au runtime (exécution du JAR)	Moyen	Script d'exécution qui résout les chemins, vérifie le JAR et initialise "HADOOP_CLASSPATH=\$(hadoop classpath)" ("scripts/execution/run_mapreduce_jobs.sh").

2.4 Planning

Phase	Durée estimée	Livrable	Statut
Recherche & choix du dataset	1-1.5 j	Cadrage + source de données	Terminé
Mise en place environnement Hadoop/Hive	2-3 j	Services opérationnels + config	Terminé
Développement & packaging MapReduce (3 jobs)	3-4 j	JAR + scripts d'exécution	Terminé
Hive (tables + requêtes + exports)	2 j	Scripts .hql + exports HDFS	Terminé
Exports locaux + visualisations + validations	1 j	CSV + PNG + scripts de test	Terminé
Rédaction du rapport	5 j	PDF	Terminé
Vidéo de démonstration	0.5 j	Fichier ou lien vidéo	Terminé

Figure 1 : Planning de réalisation du projet

3. Données (dataset) & préparation

3.1 Source du dataset

Le dataset utilisé est Individual household electric power consumption (UCI Machine Learning Repository), fichier "household_power_consumption.txt".

Il est public, documenté et stable, ce qui facilite la reproductibilité du pipeline.

Son format texte et sa série temporelle régulière sont adaptés à une ingestion HDFS et à un traitement batch Hadoop.

3.2 Volume et caractéristiques

- **Taille du fichier** : 127 MB
- **Nombre de lignes** : 2 075 260 (en-tête inclus)
- **Période couverte** : 16/12/2006 → 26/11/2010
- **Fréquence** : 1 mesure par minute

```
$ wc -l household_power_consumption.txt  
↳ 2075260 household_power_consumption.txt
```

3.3 Format et schéma des données

- Séparateur : ";"
- Date / heure : "DD/MM/YYYY" et "HH:MM:SS"
- Valeurs manquantes : "?" (converties en NULL côté Hive via serialization.null.format='?')

Le schéma est directement mappé vers une table externe Hive (consumption_raw) : deux champs temporels en "STRING" et des champs numériques en "DOUBLE".

Champ	Type (Hive)	Format / règle
Date	STRING	DD/MM/YYYY
Time	STRING	HH:MM:SS
Global_active_power	DOUBLE	décimal . ; ? possible
Global_reactive_power	DOUBLE	décimal . ; ? possible
Voltage	DOUBLE	décimal . ; ? possible
Global_intensity	DOUBLE	décimal . ; ? possible
Sub_metering_1	DOUBLE	décimal . ; ? possible
Sub_metering_2	DOUBLE	décimal . ; ? possible
Sub_metering_3	DOUBLE	décimal . ; ? possible

3.4 Préparation et nettoyage

Les traitements suivants sont appliqués au moment de la lecture (Hive et/ou jobs batch), sans "pré-traitement" lourd avant l'upload HDFS :

- Ignorer l'en-tête : Via "skip.header.line.count='1'" (Hive) et filtrage côté jobs si la ligne commence par "Date;Time;".
 - Filtrer les valeurs manquantes : Gestion de "?" (conversion en "NULL" côté Hive via "serialization.null.format=?" et exclusion des lignes côté jobs MapReduce).
 - Parsing contrôlé : Découpage par ";", suppression des espaces via ".trim()", vérification du nombre de champs attendus (minimum 3).
 - Conversion de types : Conversion des champs numériques en "DOUBLE" via "Double.parseDouble()" ; lignes non parsables ignorées et comptabilisées via compteurs Hadoop :
 - Jobs 1 et 2 : "MAPPER.INVALID_LINES", "REDUCER.INVALID_VALUES"
 - Job 3 : "MAPPER.INVALID_CONSUMPTION", "MAPPER.INVALID_DATES", "REDUCER.INVALID_VALUES"
 - Validations : Exclusion des valeurs négatives (consommation < 0) dans tous les jobs ; validation des dates uniquement dans le Job 3 (parsing "dd/MM/yyyy" via "SimpleDateFormat").
-

4. Architecture & déploiement Hadoop

4.1 Environnement de déploiement

Mode de déploiement : Pseudo-distribué sur 1 nœud (tous les services Hadoop sur la même machine).

Machine : Poste local (Lenovo Yoga Pro - Intel Core i9, 64 GB RAM).

Système d'exploitation : Linux (Ubuntu/Debian).

Ce choix est pertinent pour un projet académique : il réduit la charge d'administration tout en démontrant l'usage réel de "HDFS", "YARN", "MapReduce" et "Hive" sur une chaîne de traitement complète.

Limites : Absence de cluster multi-nœuds (pas de distribution réelle ni de tolérance aux pannes à l'échelle d'un cluster), et performances dépendantes d'une seule machine.

4.2 Versions et dépendances logicielles

Les versions retenues assurent la compatibilité entre l'exécution Hadoop et le packaging Java des jobs MapReduce.

Composant	Version	Rôle dans le projet
Hadoop	3.3.6	HDFS + YARN + runtime MapReduce
Hive	3.1.3	exécution SQL + exports sur données HDFS
Java	JDK 8	compilation/exécution des jobs MapReduce
Maven	3.6+	packaging du JAR MapReduce

Figure 3 : Versions et dépendances logicielles

4.3 Architecture des services Hadoop

L'application repose sur les services suivants (tous exécutés localement, en pseudo-distribué) :

HDFS (stockage) :

- **NameNode** : Gestion de l'espace de noms HDFS et des métadonnées (répertoires/fichiers).
- **DataNode** : Stockage des blocs de données.
- **Usage projet** : Centraliser les entrées/sorties du pipeline sous "/user/projet/..." (dataset, outputs MapReduce, exports Hive).

YARN (orchestration / supervision) :

- **ResourceManager** : allocation des ressources et suivi des applications (jobs MapReduce).
- **NodeManager** : exécution des containers sur le nœud et remontée d'état.
- **Usage projet** : prouver l'exécution des traitements batch (états, logs) et contrôler l'allocation CPU/RAM.

Hive (métadonnées + requêtes + exports) :

- **Client Hive** : exécution des scripts “.hql” via “hive -f”.
- **Metastore** : métadonnées des tables (metastore embarqué Derby par défaut, adapté à un environnement mono-nœud, j'aurais pu utiliser mysql).
- **Usage projet** : déclarer des tables externes pointant vers HDFS (données brutes et résultats MapReduce) et produire des exports en répertoires HDFS.

Interaction (flux simple) :

HDFS stocke les données → YARN exécute les jobs MapReduce qui lisent/écrivent sur HDFS → Hive référence ces emplacements via tables externes et exécute requêtes/exports → les exports HDFS sont consolidés en CSV locaux pour la restitution.

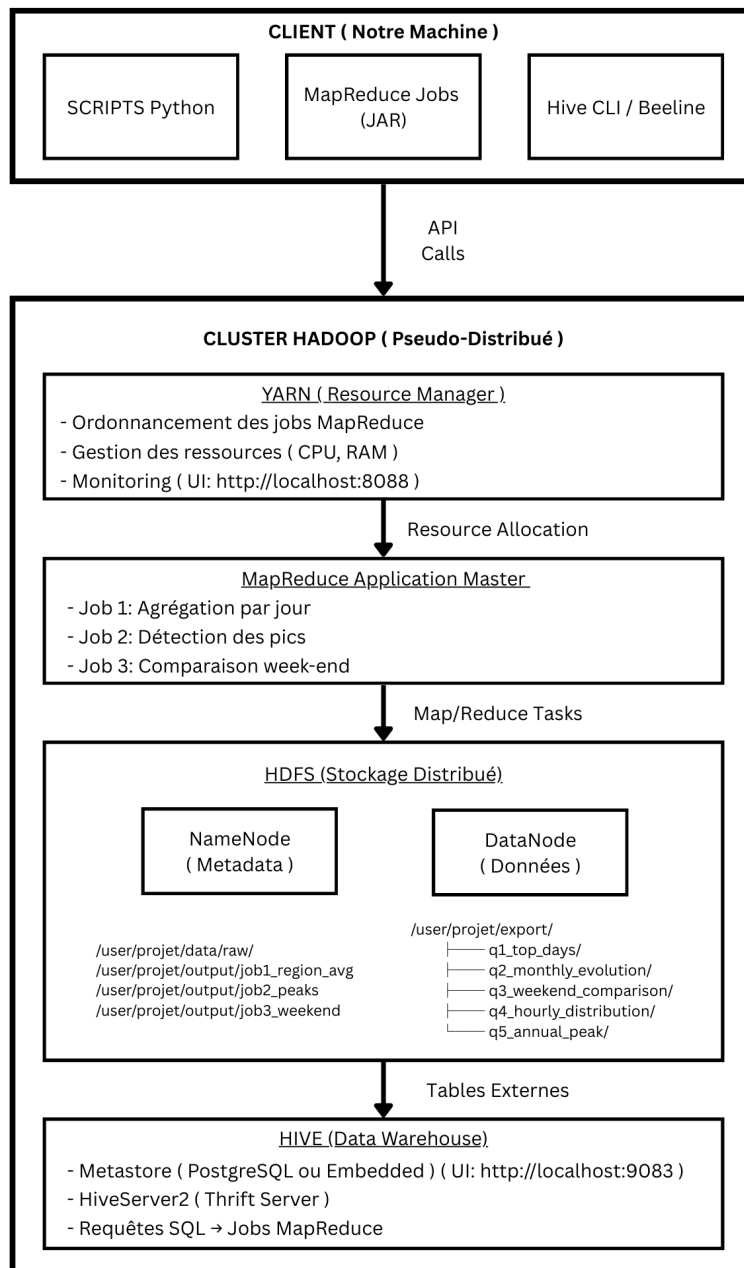


Figure 4 : Architecture du projet Hadoop pseudo-distribué

4.4 Configuration YARN (ressources)

La configuration YARN est dimensionnée pour un nœud unique afin d'éviter la sur-allocation tout en gardant des containers suffisamment grands pour exécuter les jobs. Les paramètres ci-dessous proviennent de "config/yarn-site.xml".

- **Ressources NodeManager** : 8GB, 8 vcores
- **Allocation containers** : min 1024 MB, max 8192 MB
- **ApplicationMaster MapReduce** : 1024 MB
- **Scheduler** : "CapacityScheduler"
- **Compression intermédiaire** : sorties mapper compressées (codec Snappy)

Justification : Cette configuration alloue volontairement 8 GB de RAM au NodeManager sur une machine disposant de 64 GB, ce qui est suffisant pour exécuter les 3 jobs MapReduce du projet tout en conservant des ressources pour le système d'exploitation et les autres services (HDFS, Hive). Cette approche est adaptée à un environnement de développement académique et permet d'observer l'allocation de ressources dans l'UI YARN sans risquer l'échec par manque de mémoire.

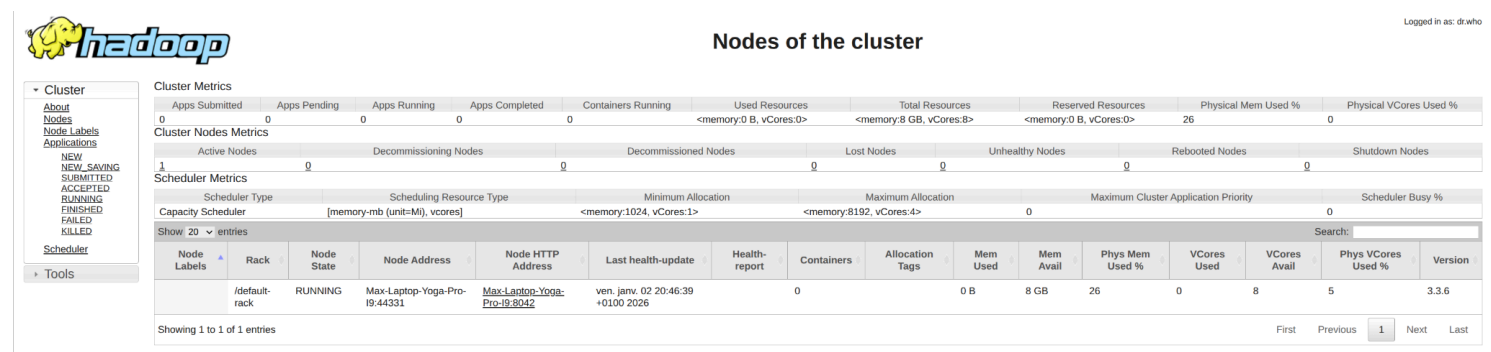


Figure 5 : UI YARN "Nodes" : <http://localhost:8088>

4.5 Variables d'environnement et accès

Les scripts du projet appellent directement "hdfs", "hadoop", "yarn" et "hive". Les variables suivantes garantissent la résolution des binaires et des bibliothèques :

- **JAVA_HOME** : nécessaire pour exécuter les démons Hadoop et les jobs Java.
- **HADOOP_HOME** : accès aux binaires et à la configuration Hadoop.
- **HIVE_HOME** : accès au client Hive et à sa configuration.
- **PATH** : inclut \$HADOOP_HOME/bin, \$HADOOP_HOME/sbin, \$HIVE_HOME/bin.

À l'exécution, le classpath Hadoop est résolu dynamiquement ("HADOOP_CLASSPATH=\$(hadoop classpath)") afin d'exécuter les drivers Java avec les dépendances Hadoop correctes.

4.6 Interfaces web et supervision

Les interfaces web servent de preuves et d'outils de validation du bon fonctionnement des services :

- **NameNode UI** : <http://localhost:9870>

```
maxiimax@Max-Laptop-Yoga-Pro-I9:~/Bureau/FAC/M1/SEMESTRE-1/Cadre logiciel pour le big data/Chapitre-7/PROJET/scripts/visualizati
on$ hdfs dfs -ls /user/projet/data/raw
Found 1 items
-rwxr-xr-x 1 maxiimax supergroup 132960755 2025-12-27 16:52 /user/projet/data/raw/household_power_consumption.txt
maxiimax@Max-Laptop-Yoga-Pro-I9:~/Bureau/FAC/M1/SEMESTRE-1/Cadre logiciel pour le big data/Chapitre-7/PROJET/scripts/visualizati
on$
```

Figure 6 : Vérification de “household_power_consumption.txt” (dataset) au bon endroit dans HDFS

```
on$ hdfs dfs -ls /user/projet/output
Found 3 items
drwxr-xr-x - maxiimax supergroup 0 2026-01-01 20:24 /user/projet/output/job1_region_avg
drwxr-xr-x - maxiimax supergroup 0 2026-01-01 20:24 /user/projet/output/job2_peaks
drwxr-xr-x - maxiimax supergroup 0 2026-01-01 20:24 /user/projet/output/job3_weekend
maxiimax@Max-Laptop-Yoga-Pro-I9:~/Bureau/FAC/M1/SEMESTRE-1/Cadre logiciel pour le big data/Chapitre-7/PROJET/scripts/visualizati
on$
```

Figure 7 : Vérification de la sauvegarde des sorties après traitement dans HDFS

Started:	Sat Jan 03 16:19:51 +0100 2026
Version:	3.3.6, r1be78238728da9266a4f88195058f08fd012bf9c
Compiled:	Sun Jun 18 10:22:00 +0200 2023 by ubuntu from (HEAD detached at release-3.3.6-RC1)
Cluster ID:	CID-590ad144-e820-4f11-9945-dafb13042fe8
Block Pool ID:	BP-1539471611-127.0.1.1-1766689130195

Summary

Security is off.

Safemode is off.

549 files and directories, 509 blocks (509 replicated blocks, 0 erasure coded block groups) = 1 058 total filesystem object(s).

Heap Memory used 471.32 MB of 1.34 GB Heap Memory. Max Heap Memory is 13.86 GB.

Non Heap Memory used 59.67 MB of 61.13 MB Committed Non Heap Memory. Max Non Heap Memory is <unbounded>.

Configured Capacity:	491.08 GB
Configured Remote Capacity:	0 B
DFS Used:	310 MB (0.06%)
Non DFS Used:	101.62 GB
DFS Remaining:	364.14 GB (74.15%)
Block Pool Used:	310 MB (0.06%)
DataNodes usages% (Min/Median/Max/stdDev):	0.06% / 0.06% / 0.06% / 0.00%
Live Nodes	1 (Decommissioned: 0, In Maintenance: 0)
Dead Nodes	0 (Decommissioned: 0, In Maintenance: 0)
Decommissioning Nodes	0
Entering Maintenance Nodes	0
Total Datanode Volume Failures	0 (0 B)
Number of Under-Replicated Blocks	4
Number of Blocks Pending Deletion (including replicas)	0
Block Deletion Start Time	Sat Jan 03 16:19:51 +0100 2026
Last Checkpoint Time	Sat Jan 03 16:19:52 +0100 2026
Enabled Erasure Coding Policies	RS-6-3-1024k

Figure 8 : **NameNode UI** : <http://localhost:9870>

- **YARN ResourceManager UI** : <http://localhost:8088>

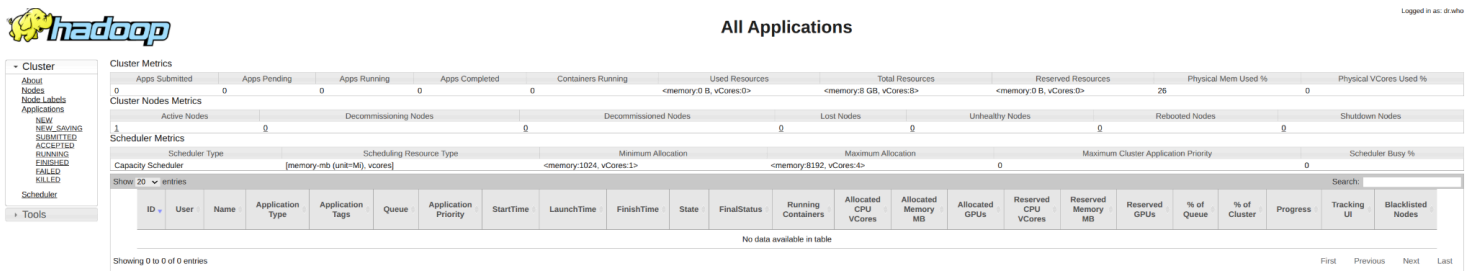


Figure 9 : YARN ResourceManager UI : <http://localhost:8088>

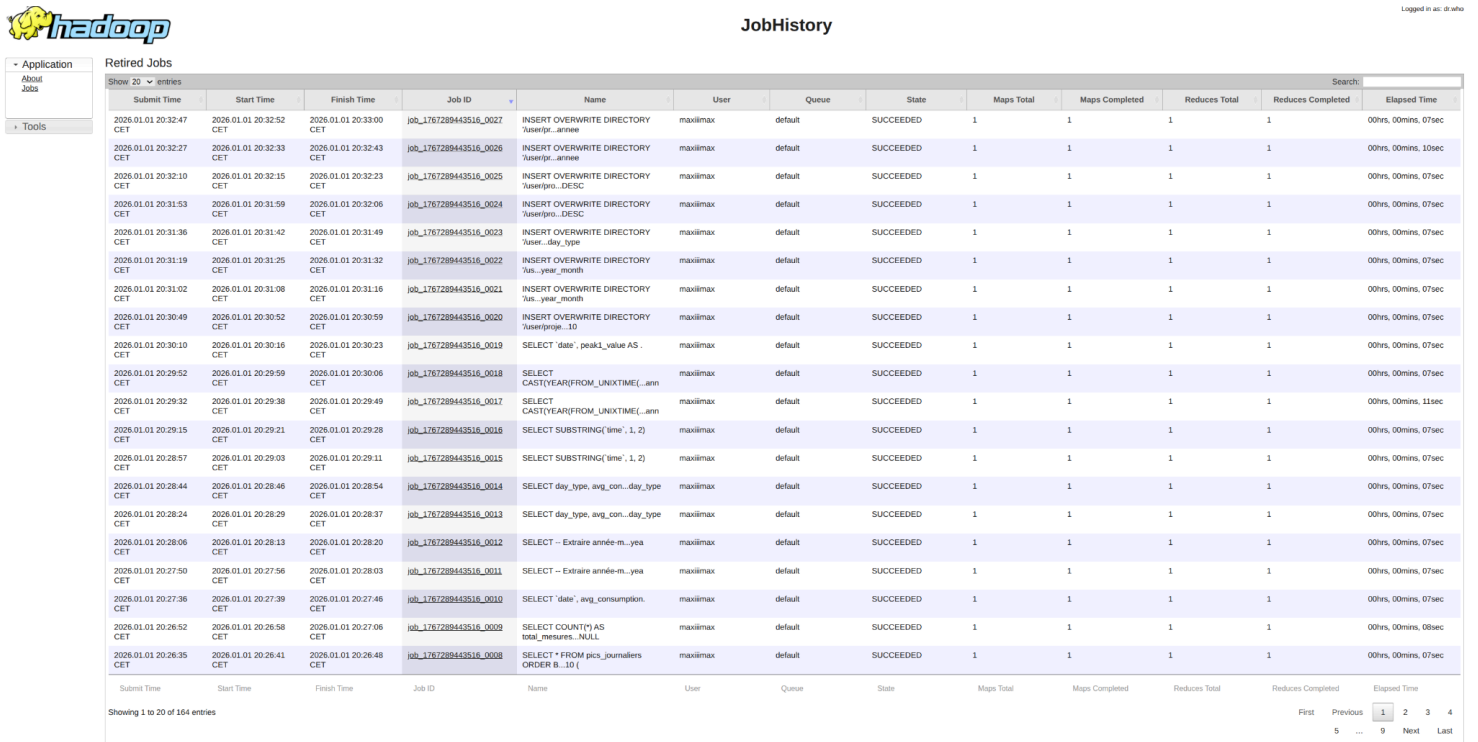


Figure 10 : Interface graphique YARN JobHistory : <http://localhost:19888> (mapred --daemon start historyserver)

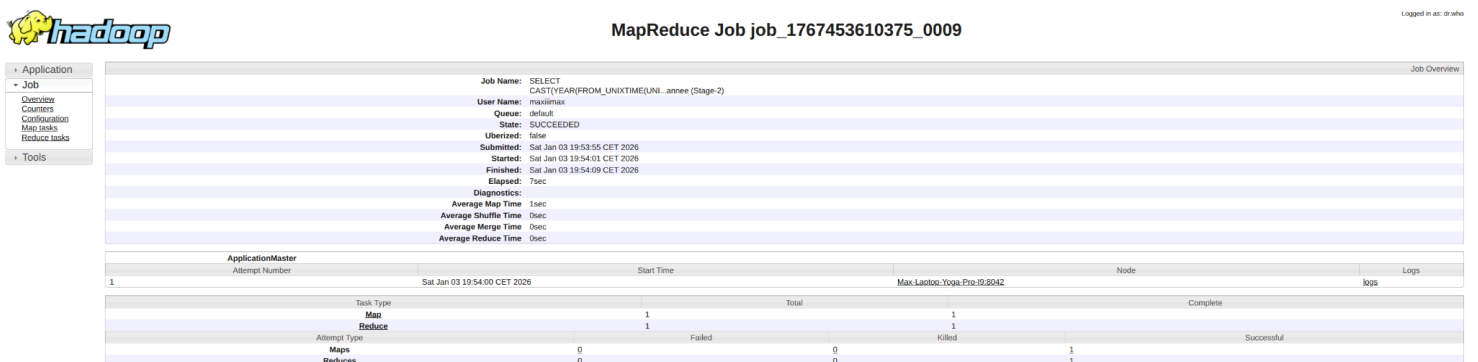


Figure 11 : Détail d'un job MapReduce - Interface JobHistory Server

5. Ingestion HDFS & organisation des données

5.1 Organisation HDFS du projet

L'arborescence HDFS est isolée sous “/user/projet” afin de séparer clairement données, résultats et exports :

- **/user/projet/data/raw** : données brutes (entrée du pipeline).
- **/user/projet/data/processed** : zone optionnelle de préparation.
- **/user/projet/output/** : sorties MapReduce, un répertoire par job (job1_region_avg, job2_peaks, job3_weekend).
- **/user/projet/export/** : exports Hive, un répertoire par requête (q1_top_days ... q5_annual_peak).

5.2 Création de la structure HDFS

La structure est créée via le script “scripts/hdfs/create_structure.sh”, qui vérifie l'accès à HDFS puis crée les répertoires de manière idempotente (création si absent).

```
cd scripts/hdfs
./create_structure.sh
```

Preuve (répertoires racine) :

```
maxiimax@Max-Laptop-Yoga-Pro-19:~/Bureau/FAC/M1/SEMESTRE-1/Cadre logiciel pour le big data/Chapitre-7/PROJET/scripts/visualizati
on$ hdfs dfs -ls /user/projet/
Found 3 items
drwxr-xr-x - maxiimax supergroup          0 2025-12-27 16:51 /user/projet/data
drwxr-xr-x - maxiimax supergroup          0 2025-12-27 18:20 /user/projet/export
drwxr-xr-x - maxiimax supergroup          0 2026-01-01 20:24 /user/projet/output
```

Figure 12 : Preuve repertoires racine

5.3 Chargement des données

Commande d'ingestion (fichier local → HDFS) :

```
hdfs dfs -put household_power_consumption.txt /user/projet/data/raw/
```

Vérification (présence + taille) :

```
$ hdfs dfs -ls /user/projet/data/raw/household_power_consumption.txt
```

L'ingestion est considérée réussie lorsque le fichier est visible dans HDFS au bon emplacement.

```
maxiimax@Max-Laptop-Yoga-Pro-19:~/Bureau/FAC/M1/SEMESTRE-1/Cadre logiciel pour le big data/Chapitre-7/PROJET/scripts/visualizati
on$ hdfs dfs -ls /user/projet/data/raw
Found 1 items
-rwxr-xr-x  1 maxiimax supergroup 132960755 2025-12-27 16:52 /user/projet/data/raw/household_power_consumption.txt
```


6. Développement MapReduce (sous YARN)

6.1 Build et packaging des jobs

Les traitements sont implémentés en Java et packagés avec Maven. Le build produit un JAR unique contenant les drivers, mappers et reducers ; cet artefact est ensuite utilisé pour lancer les jobs sur le cluster YARN.

- **Script de build** : "scripts/execution/build_mapreduce.sh" (wrapper Maven)
- **Commande** :

```
cd mapreduce
mvn clean package
```

- **Artefact généré** : "mapreduce/target/mapreduce-consumption-1.0.jar"

6.2 Exécution des jobs MapReduce

L'exécution est orchestrée par "scripts/execution/run_mapreduce_jobs.sh". Le script :

- Vérifie la disponibilité des commandes Hadoop et l'accès à HDFS.
- Vérifie la présence du répertoire d'entrée HDFS.
- Enchaîne les jobs dans un ordre déterministe et affiche un extrait des sorties en fin de job (validation rapide).

Commande (exécution standard) :

```
cd scripts/execution
./run_mapreduce_jobs.sh ../../mapreduce/target/mapreduce-consumption-1.0.jar
```

Entrée/sorties principales (valeurs par défaut du script) :

- **Input HDFS** : "/user/projet/data/raw"
- **Output base HDFS** : "/user/projet/output" (répertoires job1_region_avg, job2_peaks, job3_weekend)

6.3 Description des traitements MapReduce

- **Job 1 — Agrégation journalière (RegionAvgDriver)** : Calcule, pour chaque date, des statistiques sur Global_active_power. Le Mapper émet (date, puissance), le Reducer agrège en (date, avg,min,max,count).
- **Format de sortie** : date<TAB>avg,min,max,count.
- **Preuve (extrait HDFS)** :

```
maxiilmax@Max-Laptop-Yoga-Pro-I9:~/Bureau/FAC/M1/SEMESTRE-1/Cadre logiciel pour le big data/Chapitre-7/PROJET/scripts/visualizati
on$ hdfs dfs -cat /user/projet/output/job1_region_avg/part-r-000000 | head -5
1/1/2007      1.9090,0.2040,3.5580,1440
1/1/2008      1.9165,0.2260,5.6860,1440
1/1/2009      0.9769,0.2040,8.5840,1440
1/1/2010      0.8502,0.3220,4.9760,1440
1/10/2007     1.1951,0.1940,7.6240,1440
```

Figure 13 : Preuve d'exécution Job 1 - Extrait des résultats HDFS

- **Job 2 — Détection des pics journaliers (PeakDetectionDriver)** : Extrait, pour chaque date, les trois pics de consommation (heure + valeur). Le Mapper émet (date, time,puissance), le Reducer conserve les top 3 valeurs pour produire une sortie compacte.
- **Format de sortie** : date<TAB>time1,val1,time2,val2,time3,val3.
- **Preuve (extrait HDFS)** :

```
maxiimax@Max-Laptop-Yoga-Pro-I9:~/Bureau/FAC/M1/SEMESTRE-1/Cadre logiciel pour le big data/Chapitre-7/PROJET/scripts/visualizati
on$ hdfs dfs -cat /user/projet/output/job2_peaks/part-r-000000 | head -5
1/1/2007      09:50:00,3.5580,09:49:00,3.5420,09:51:00,3.5420
1/1/2008      11:38:00,5.6860,18:02:00,5.5400,18:01:00,5.5060
1/1/2009      20:50:00,8.5840,20:49:00,7.8200,20:47:00,7.0540
1/1/2010      16:59:00,4.9760,17:00:00,4.9680,16:46:00,4.4400
1/10/2007     09:41:00,7.6240,09:42:00,7.5640,09:43:00,7.5080
```

Figure 14 : Sortie Job 2 - Détection des pics journaliers (extrait HDFS)

- **Job 3 — Comparaison semaine / week-end (WeekendComparisonDriver)** : Regroupe les mesures selon "day_type ∈ {weekday, weekend}" (déduit de la date). Le Mapper émet (day_type, puissance), le Reducer calcule (avg,min,max,count) par groupe.
- **Format de sortie** : day_type<TAB>avg,min,max,count.
- **Preuve (extrait HDFS)** :

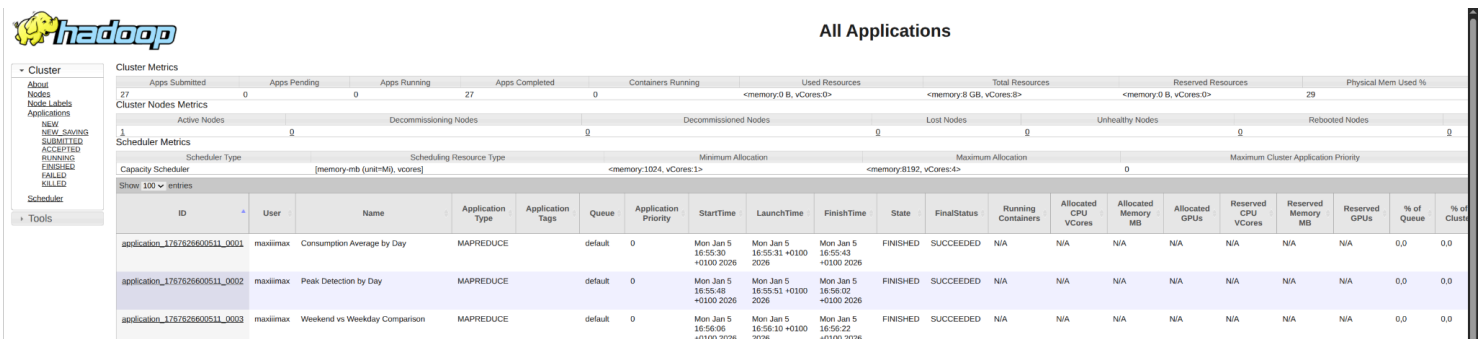
```
maxiimax@Max-Laptop-Yoga-Pro-I9:~/Bureau/FAC/M1/SEMESTRE-1/Cadre logiciel pour le big data/Chapitre-7/PROJET/scripts/visualizati
on$ hdfs dfs -cat /user/projet/output/job3_weekend/part-r-000000
weekday 1.0355,0.0760,9.7320,1470428
weekend 1.2342,0.0780,11.1220,578852
```

Figure 15 : Sortie Job 3 - Comparaison semaine / week-end (extrait HDFS)

6.4 Exécution sous YARN et suivi

L'exécution MapReduce s'appuie sur YARN, le ResourceManager planifie les applications et le NodeManager exécute les containers sur le nœud (pseudo-distribué). Chaque job correspond à une application YARN distincte.

- **Captures à insérer (UI YARN ResourceManager <http://localhost:8088>)** :



The screenshot shows the Hadoop YARN ResourceManager web interface. On the left is a navigation menu with options like Cluster, About, Nodes, Node Labels, Applications, and Scheduler. The main area is titled 'All Applications' and displays a table of application metrics. The table has columns for ID, User, Name, Application Type, Application Tags, Queue, Application Priority, Start Time, Launch Time, Finish Time, State, Final Status, Running Containers, Allocated CPU Vcores, Allocated Memory MB, Allocated GPUs, Reserved CPU Vcores, Reserved Memory MB, Reserved GPUs, % of Queue, and % of Cluster. Three applications are listed: 'Consumption Average by Day', 'Peak Detection by Day', and 'Weekend vs Weekday Comparison', all of which are in a 'FINISHED' state with a 'SUCCEEDED' final status.

ID	User	Name	Application Type	Application Tags	Queue	Application Priority	StartTime	LaunchTime	FinishTime	State	FinalStatus	Running Containers	Allocated CPU Vcores	Allocated Memory MB	Allocated GPUs	Reserved CPU Vcores	Reserved Memory MB	Reserved GPUs	% of Queue	% of Cluster
application_176752960511_0001	maxiimax	Consumption Average by Day	MAPREDUCE		default	0	Mon Jan 5 16:55:30 +0100 2026	Mon Jan 5 16:55:31 +0100 2026	Mon Jan 5 16:55:43 +0100 2026	FINISHED	SUCCEEDED	N/A	N/A	N/A	N/A	N/A	N/A	N/A	0.0	0.0
application_176752960511_0002	maxiimax	Peak Detection by Day	MAPREDUCE		default	0	Mon Jan 5 16:55:48 +0100 2026	Mon Jan 5 16:55:51 +0100 2026	Mon Jan 5 16:56:02 +0100 2026	FINISHED	SUCCEEDED	N/A	N/A	N/A	N/A	N/A	N/A	N/A	0.0	0.0
application_176752960511_0003	maxiimax	Weekend vs Weekday Comparison	MAPREDUCE		default	0	Mon Jan 5 16:56:00 +0100 2026	Mon Jan 5 16:56:10 +0100 2026	Mon Jan 5 16:56:22 +0100 2026	FINISHED	SUCCEEDED	N/A	N/A	N/A	N/A	N/A	N/A	N/A	0.0	0.0

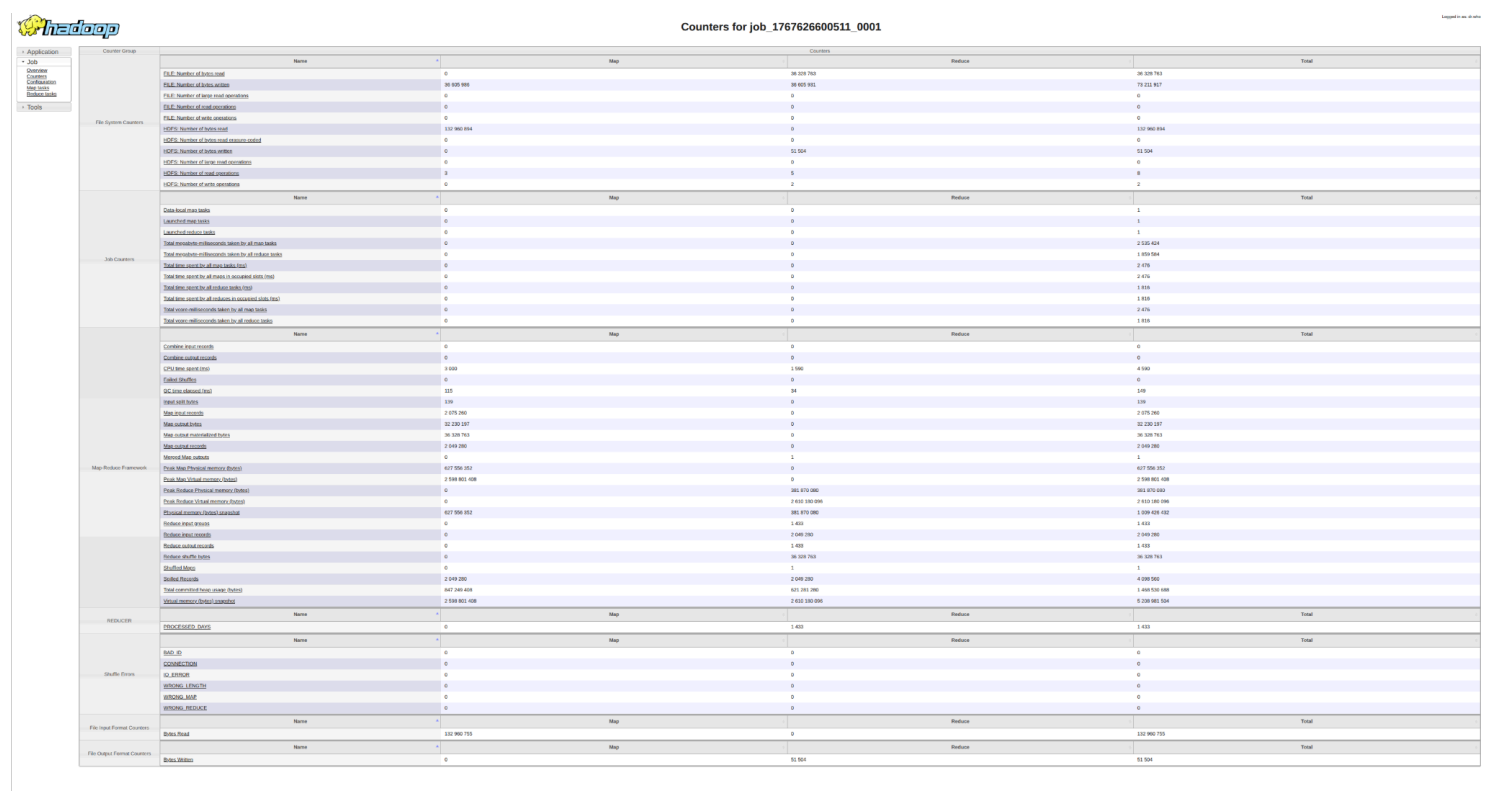
Figure 16 : Interface YARN ResourceManager - Liste des applications MapReduce

6.5 Gestion des erreurs et validations

Les traitements intègrent des contrôles simples au niveau parsing, et des validations de présence de sorties :

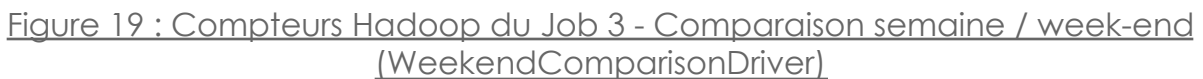
- **Entrées invalides / valeurs manquantes** : Lignes rejetées si en-tête, si champs manquants, ou si valeurs “?” sur les colonnes requises.
- **Formats incorrects** : Erreurs de conversion numérique capturées (ex. NumberFormatException).
- **Dates** : Contrôle spécifique dans le job week-end (dates invalides comptabilisées).

Compteurs Hadoop utilisés (preuves via UI YARN / logs) :



Category	Name	Map	Reduce	Total
Core Counters	FileL: Number of bytes read	0	36 529 743	36 529 743
	FileL: Number of bytes written	0	132 960 894	132 960 894
	FileL: Number of bytes read (compressed)	0	0	0
	FileL: Number of bytes written (compressed)	0	0	0
	FileL: Number of bytes read (uncompressed)	0	0	0
	FileL: Number of bytes written (uncompressed)	0	0	0
File System Counters	FileS: Number of bytes read	0	132 960 894	132 960 894
	FileS: Number of bytes read (compressed)	0	0	0
	FileS: Number of bytes read (uncompressed)	0	0	0
	FileS: Number of bytes written	0	132 960 894	132 960 894
	FileS: Number of bytes written (compressed)	0	0	0
	FileS: Number of bytes written (uncompressed)	0	0	0
Job Counters	Job: Map input records	0	0	0
	Job: Map input bytes	0	0	0
	Job: Map output records	0	0	0
	Job: Map output bytes	0	0	0
	Job: Reduce input records	0	0	0
	Job: Reduce input bytes	0	0	0
Map Counters	Map: Input records	0	0	0
	Map: Input bytes	0	0	0
	Map: Output records	0	0	0
	Map: Output bytes	0	0	0
	Map: Spill records	0	0	0
	Map: Spill bytes	0	0	0
MapReduce Counters	MapReduce: Map input records	0	0	0
	MapReduce: Map input bytes	0	0	0
	MapReduce: Map output records	0	0	0
	MapReduce: Map output bytes	0	0	0
	MapReduce: Reduce input records	0	0	0
	MapReduce: Reduce input bytes	0	0	0
Shuffle Counters	Shuffle: Map input records	0	0	0
	Shuffle: Map input bytes	0	0	0
	Shuffle: Map output records	0	0	0
	Shuffle: Map output bytes	0	0	0
	Shuffle: Reduce input records	0	0	0
	Shuffle: Reduce input bytes	0	0	0
File Input Counters	FileInput: Map input records	0	0	0
	FileInput: Map input bytes	0	0	0
	FileInput: Map output records	0	0	0
	FileInput: Map output bytes	0	0	0
	FileInput: Reduce input records	0	0	0
	FileInput: Reduce input bytes	0	0	0
File Output Counters	FileOutput: Map input records	0	0	0
	FileOutput: Map input bytes	0	0	0
	FileOutput: Map output records	0	0	0
	FileOutput: Map output bytes	0	0	0
	FileOutput: Reduce input records	0	0	0
	FileOutput: Reduce input bytes	0	0	0

Figure 17 : Page "Counters" job1 MapReduce (compteurs système et framework)





JobHistory

Logged in as: dr.salto

Application		Retired Jobs										
About Jobs		Search:										
Tools												
Show 20 entries												
Submit Time	Start Time	Finish Time	Job ID	Name	User	Queue	State	Maps Total	Maps Completed	Reduces Total	Reduces Completed	Elapsed Time
2026.01.01 20:32:47 CET	2026.01.01 20:32:52 CET	2026.01.01 20:33:00 CET	job_1767289443516_0027	INSERT OVERWRITE DIRECTORY 'user/p...annee	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 07sec
2026.01.01 20:32:27 CET	2026.01.01 20:32:33 CET	2026.01.01 20:32:43 CET	job_1767289443516_0026	INSERT OVERWRITE DIRECTORY 'user/p...annee	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 10sec
2026.01.01 20:32:10 CET	2026.01.01 20:32:15 CET	2026.01.01 20:32:23 CET	job_1767289443516_0025	INSERT OVERWRITE DIRECTORY 'user/p...DESC	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 07sec
2026.01.01 20:31:53 CET	2026.01.01 20:31:59 CET	2026.01.01 20:32:06 CET	job_1767289443516_0024	INSERT OVERWRITE DIRECTORY 'user/p...DESC	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 07sec
2026.01.01 20:31:36 CET	2026.01.01 20:31:42 CET	2026.01.01 20:31:49 CET	job_1767289443516_0023	INSERT OVERWRITE DIRECTORY 'user...day_type	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 07sec
2026.01.01 20:31:19 CET	2026.01.01 20:31:25 CET	2026.01.01 20:31:32 CET	job_1767289443516_0022	INSERT OVERWRITE DIRECTORY 'us...year_month	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 07sec
2026.01.01 20:31:02 CET	2026.01.01 20:31:08 CET	2026.01.01 20:31:16 CET	job_1767289443516_0021	INSERT OVERWRITE DIRECTORY 'us...year_month	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 07sec
2026.01.01 20:30:49 CET	2026.01.01 20:30:52 CET	2026.01.01 20:30:59 CET	job_1767289443516_0020	INSERT OVERWRITE DIRECTORY 'user/proje...10	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 07sec
2026.01.01 20:30:10 CET	2026.01.01 20:30:16 CET	2026.01.01 20:30:23 CET	job_1767289443516_0019	SELECT 'date', peak1_value AS .	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 07sec
2026.01.01 20:29:52 CET	2026.01.01 20:29:59 CET	2026.01.01 20:30:06 CET	job_1767289443516_0018	SELECT CAST(YEAR(FROM_UNIXTIME(...ann	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 07sec
2026.01.01 20:29:32 CET	2026.01.01 20:29:38 CET	2026.01.01 20:29:49 CET	job_1767289443516_0017	SELECT CAST(YEAR(FROM_UNIXTIME(...ann	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 11sec
2026.01.01 20:29:15 CET	2026.01.01 20:29:21 CET	2026.01.01 20:29:28 CET	job_1767289443516_0016	SELECT SUBSTRING('time', 1, 2)	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 07sec
2026.01.01 20:28:57 CET	2026.01.01 20:29:03 CET	2026.01.01 20:29:11 CET	job_1767289443516_0015	SELECT SUBSTRING('time', 1, 2)	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 07sec
2026.01.01 20:28:44 CET	2026.01.01 20:28:46 CET	2026.01.01 20:28:54 CET	job_1767289443516_0014	SELECT day_type, avg_con...day_type	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 07sec
2026.01.01 20:28:24 CET	2026.01.01 20:28:29 CET	2026.01.01 20:28:37 CET	job_1767289443516_0013	SELECT day_type, avg_con...day_type	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 07sec
2026.01.01 20:28:06 CET	2026.01.01 20:28:13 CET	2026.01.01 20:28:20 CET	job_1767289443516_0012	SELECT -- Extraire année-m...yea	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 07sec
2026.01.01 20:27:50 CET	2026.01.01 20:27:56 CET	2026.01.01 20:28:03 CET	job_1767289443516_0011	SELECT -- Extraire année-m...yea	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 07sec
2026.01.01 20:27:36 CET	2026.01.01 20:27:39 CET	2026.01.01 20:27:46 CET	job_1767289443516_0010	SELECT 'date', avg_consumption.	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 07sec
2026.01.01 20:26:52 CET	2026.01.01 20:26:58 CET	2026.01.01 20:27:06 CET	job_1767289443516_0009	SELECT COUNT(*) AS total_mesures...NULL	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 08sec
2026.01.01 20:26:35 CET	2026.01.01 20:26:41 CET	2026.01.01 20:26:48 CET	job_1767289443516_0008	SELECT * FROM pics_journaliers ORDER B...10 (	maximax	default	SUCCEEDED	1	1	1	1	00hrs, 00mins, 07sec
Submit Time	Start Time	Finish Time	Job ID	Name	User	Queue	State	Maps Total	Maps Completed	Reduces Total	Reduces Completed	Elapsed Time
Showing 1 to 20 of 164 entries												
												First Previous 1 2 3 4 5 ... 9 Next Last

Figure 20 : Interface JobHistory Server - Liste des applications terminées (Retired Jobs)

7. Data Warehouse Hive : schéma + requêtes analytiques

7.1 Création de la base de données

La base Hive utilisée est "consommation_elec". Elle sert de namespace pour regrouper tables et requêtes, tout en laissant les données physiques sur HDFS (données brutes et outputs sous "/user/projet/...").

Création via "hive/scripts/01_create_database.hql" :

```
CREATE DATABASE IF NOT EXISTS consommation_elec

LOCATION '/user/hive/warehouse/consommation_elec.db';

USE consommation_elec;
```

7.2 Modélisation des tables Hive

Le projet s'appuie principalement sur des tables externes, Hive référence des répertoires HDFS existants (données brutes et sorties MapReduce) sans recopier les fichiers. Cela permet de rejouer le pipeline (jobs ré-exécutés → répertoires HDFS mis à jour) sans gestion de chargement interne côté Hive.

La table "consumption_raw" référence le dataset brut dans "/user/projet/data/raw" (TEXTFILE ;). Les propriétés "skip.header.line.count" et "serialization.null.format" garantissent une lecture robuste (en-tête ignoré, ? → NULL).

```
CREATE EXTERNAL TABLE consumption_raw (

  `date` STRING,
  `time` STRING,
  global_active_power DOUBLE,
  global_reactive_power DOUBLE,
  voltage DOUBLE,
  global_intensity DOUBLE,
  sub_metering_1 DOUBLE,
  sub_metering_2 DOUBLE,
  sub_metering_3 DOUBLE

)

ROW FORMAT DELIMITED FIELDS TERMINATED BY ','
STORED AS TEXTFILE
LOCATION '/user/projet/data/raw'
TBLPROPERTIES ('skip.header.line.count'='1', 'serialization.null.format'='?');
```

Figure 21 : Définition de la table Hive consumption_raw (CREATE EXTERNAL TABLE)
(02_create_tables.hql)

Les outputs MapReduce sont exposés via des tables externes pointant vers “/user/projet/output/...” (format TEXTFILE). Les schémas correspondent aux agrégations calculées par les jobs :

- **conso_par_jour (Job 1)** : “date, avg_consumption”, “min_consumption”, “max_consumption”, “count” → “/user/projet/output/job1_region_avg”
- **pics_journaliers (Job 2)** : “date, peak1_time/value”, “peak2_time/value”, “peak3_time/value” → “/user/projet/output/job2_peaks”
- **comparaison_jours (Job 3)** : “day_type”, “avg_consumption”, “min_consumption”, “max_consumption”, “count” → “/user/projet/output/job3_weekend”

Point d'attention (format MapReduce → Hive) : Les sorties Job 1 et Job 3 sont de la forme “cle<TAB>valeurs_csv”. Les tables “conso_par_jour” et “comparaison_jours” utilisent “FIELDS TERMINATED BY ','” et incluent la clé (date/day_type) comme première colonne, malgré le séparateur TAB entre la clé et les valeurs dans les fichiers MapReduce. Ce désalignement est documenté dans “02_create_tables.hql” (lignes 55-56) et produit un champ NULL supplémentaire (“\N”) en fin de ligne dans les exports CSV. Les requêtes Q1, Q2 et Q3 fonctionnent malgré ce problème car Hive parvient à extraire les valeurs nécessaires, bien que l'alignement des colonnes ne soit pas optimal.

```
maxiimax@Max-Laptop-Yoga-Pro-I9:~/Bureau/FAC/M1/SEMESTRE-1/Cadre logiciel pour le big data/Chapitre-7/PROJET/scripts/visualizati
on$ hive -e "USE consommation_elec; SHOW TABLES;"
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/home/maxiimax/hive/lib/log4j-slf4j-impl-2.17.1.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/home/maxiimax/hadoop-3.3.6/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/org/slf4j/impl/
StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
Hive Session ID = d05fde76-6013-4ede-bded-ee7d20dacd35

Logging initialized using configuration in jar:file:/home/maxiimax/hive/lib/hive-common-3.1.3.jar!/hive-log4j2.properties Async:
true
Hive Session ID = a7202b49-a3ba-4eea-ac06-db3ef917899b
OK
Time taken: 0.255 seconds
OK
comparaison_jours
conso_par_jour
conso_par_jour_raw
consumption_raw
pics_journaliers
Time taken: 0.081 seconds, Fetched: 5 row(s)
maxiimax@Max-Laptop-Yoga-Pro-I9:~/Bureau/FAC/M1/SEMESTRE-1/Cadre logiciel pour le big data/Chapitre-7/PROJET/scripts/visualizati
on$
```

Figure 22 : Liste des tables Hive dans la base consommation_elec (SHOW TABLES)

```

maxiimax@Max-Laptop-Yoga-Pro-I9:~/Bureau/FAC/M1/SEMESTRE-1/Cadre logiciel pour le big data/Chapitre-7/PROJET/scripts/visualizati
on$ hive -e "USE consommation_elec; DESCRIBE FORMATTED consumption_raw;"
SLF4J: Class path contains multiple SLF4J bindings.
SLF4J: Found binding in [jar:file:/home/maxiimax/hive/lib/log4j-slf4j-impl-2.17.1.jar!/org/slf4j/impl/StaticLoggerBinder.class]
SLF4J: Found binding in [jar:file:/home/maxiimax/hadoop-3.3.6/share/hadoop/common/lib/slf4j-reload4j-1.7.36.jar!/org/slf4j/impl/
StaticLoggerBinder.class]
SLF4J: See http://www.slf4j.org/codes.html#multiple_bindings for an explanation.
SLF4J: Actual binding is of type [org.apache.logging.slf4j.Log4jLoggerFactory]
Hive Session ID = 39e32b7e-93b0-496d-9f90-e68368383b97

Logging initialized using configuration in jar:file:/home/maxiimax/hive/lib/hive-common-3.1.3.jar!/hive-log4j2.properties Async:
true
Hive Session ID = 8ae2b97a-2798-4417-9e42-514de1f788f9
OK
Time taken: 0.239 seconds
OK
# col_name                data_type                comment
date                      string                   Date de la mesure (DD/MM/YYYY)
time                     string                   Heure de la mesure (HH:MM:SS)
global_active_power      double                   Puissance active globale (kilowatts)
global_reactive_power    double                   Puissance réactive globale (kilowatts)
voltage                  double                   Tension moyenne (volts)
global_intensity         double                   Intensité globale moyenne (ampères)
sub_metering_1           double                   Sous-compteur 1 - Cuisine (watt-heure)
sub_metering_2           double                   Sous-compteur 2 - Lave-linge/Climatisation (watt-heure)
sub_metering_3           double                   Sous-compteur 3 - Chauffe-eau/Climatisation (watt-heure)

# Detailed Table Information
Database:                consommation_elec
OwnerType:                USER
Owner:                   maxiimax
CreateTime:              Thu Jan 01 20:25:23 CET 2026
LastAccessTime:          UNKNOWN
Retention:               0
Location:                hdfs://localhost:9000/user/projet/data/raw
Table Type:              EXTERNAL_TABLE
Table Parameters:
    EXTERNAL              TRUE
    bucketing_version     2
    comment               Table externe contenant les données brutes de consommation électrique
    numFiles              1
    serialization.null.format ?
    skip.header.line.count 1
    totalSize             132960755
    transient_lastDdlTime 1767295523

# Storage Information
SerDe Library:            org.apache.hadoop.hive.serde2.lazy.LazySimpleSerDe
InputFormat:              org.apache.hadoop.mapred.TextInputFormat
OutputFormat:             org.apache.hadoop.hive.ql.io.HiveIgnoreKeyTextOutputFormat
Compressed:               No
Num Buckets:              -1
Bucket Columns:           []
Sort Columns:             []
Storage Desc Params:
    field.delim           ;
    serialization.format   ;
Time taken: 0.188 seconds, Fetched: 41 row(s)
maxiimax@Max-Laptop-Yoga-Pro-I9:~/Bureau/FAC/M1/SEMESTRE-1/Cadre logiciel pour le big data/Chapitre-7/PROJET/scripts/visualizati
on$

```

Figure 23 : Description détaillée de la table Hive consommation_raw (DESCRIBE FORMATTED)

7.3 Requêtes analytiques

Les requêtes sont regroupées dans "hive/queries/04_requetes_analytiques.hql". Elles s'exécutent dans la base "consommation_elec" et s'appuient sur les tables externes définies en 7.2.

- Q1 - Top 10 jours (statistiques journalières) :

Tables : "conso_par_jour". Logique : tri décroissant sur "avg_consumption" et limitation à 10 lignes


```

Stage-Stage-1: Map: 1 Reduce: 1 Cumulative CPU: 2.64 sec HDFS Read: 63575 HDFS Write: 590 SUCCESS
Total MapReduce CPU Time Spent: 2 seconds 640 msec
OK
28/9/2010      1.6391  0.842  4.108  287.0  NULL
29/10/2008     1.1198  0.66   5.042  1440.0 NULL
24/11/2008     2.2312  0.64   7.73   1440.0 NULL
27/12/2008     2.0640  0.61   5.548  1440.0 NULL
3/3/2008       0.7226  0.488  2.92   1440.0 NULL
28/4/2007      1.0828  0.488  1.478  21.0   NULL
1/3/2008       0.7930  0.478  4.038  1440.0 NULL
1/11/2008      0.5767  0.418  1.672  1440.0 NULL
2/11/2008      0.6324  0.418  2.29   1440.0 NULL
26/12/2006     2.7320  0.418  6.496  1440.0 NULL
Time taken: 15.778 seconds, Fetched: 10 row(s)
hive>

```

Figure 24 : Résultats de la requête Q1 - Top 10 jours (statistiques journalières)

```

USE consommation_elec;

SELECT
  `date`,
  avg_consumption,
  min_consumption,
  max_consumption,
  `count` AS nombre_mesures
FROM conso_par_jour
ORDER BY avg_consumption DESC
LIMIT 10;

```

Figure 25 : Requête Hive Q1 - Top 10 jours (statistiques journalières)

- Q2 - Évolution mensuelle

Tables : "conso_par_jour". Logique : construction d'un year_month à partir de date (format dd/MM/yyyy) puis agrégations mensuelles.

```

SELECT

CONCAT(
  YEAR(FROM_UNIXTIME(UNIX_TIMESTAMP(`date`, 'dd/MM/yyyy'))),
  '-',
  LPAD(MONTH(FROM_UNIXTIME(UNIX_TIMESTAMP(`date`, 'dd/MM/yyyy'))), 2, '0')
) AS year_month,

AVG(avg_consumption) AS consommation_moyenne,
SUM(`count`) AS total_mesures

FROM conso_par_jour
WHERE `date` IS NOT NULL
  AND UNIX_TIMESTAMP(`date`, 'dd/MM/yyyy') IS NOT NULL
GROUP BY CONCAT(
  YEAR(FROM_UNIXTIME(UNIX_TIMESTAMP(`date`, 'dd/MM/yyyy'))),
  '-',
  LPAD(MONTH(FROM_UNIXTIME(UNIX_TIMESTAMP(`date`, 'dd/MM/yyyy'))), 2, '0')
)

ORDER BY year_month;

```

Figure 26 : Requête Hive Q2 - Évolution mensuelle de la consommation

```

OK
2006-12 0.22237500000000002 NULL
2007-01 0.22006451612903233 NULL
2007-02 0.22350000000000003 NULL
2007-03 0.21709677419354842 NULL
2007-04 0.17696551724137932 NULL
2007-05 0.17200000000000004 NULL
2007-06 0.13146666666666668 NULL
2007-07 0.12058064516129033 NULL
2007-08 0.10419354838709678 NULL
2007-09 0.12653333333333336 NULL
2007-10 0.216 NULL
2007-11 0.21700000000000003 NULL
2007-12 0.2202580645161291 NULL
2008-01 0.26406451612903226 NULL
2008-02 0.21455172413793108 NULL
2008-03 0.2347096774193549 NULL
2008-04 0.24446666666666667 NULL
2008-05 0.1872258064516129 NULL
2008-06 0.14833333333333337 NULL
2008-07 0.152 NULL
2008-08 0.08541935483870965 NULL
2008-09 0.15839999999999996 NULL
2008-10 0.24806451612903233 NULL
2008-11 0.2666 NULL
2008-12 0.23935483870967747 NULL
2009-01 0.23212903225806453 NULL
2009-02 0.2175714285714286 NULL
2009-03 0.23258064516129032 NULL
2009-04 0.2778 NULL
2009-05 0.23716129032258065 NULL
2009-06 0.17117241379310347 NULL
2009-07 0.14670967741935484 NULL
2009-08 0.14593548387096777 NULL
2009-09 0.15113333333333331 NULL
2009-10 0.20993548387096775 NULL
2009-11 0.225866666666666672 NULL
2009-12 0.23935483870967741 NULL
2010-01 0.250866666666666674 NULL
2010-02 0.24942857142857147 NULL
2010-03 0.24793548387096773 NULL
2010-04 0.23573333333333338 NULL
2010-05 0.2280645161290323 NULL
2010-06 0.17099999999999999 NULL
2010-07 0.1758709677419355 NULL
2010-08 0.16044444444444445 NULL
2010-09 0.18350000000000002 NULL
2010-10 0.25567741935483873 NULL
2010-11 0.23492307692307687 NULL
Time taken: 30.979 seconds, Fetched: 48 row(s)
hive>

```

Figure 27 : Résultats de la requête Q2 - Évolution mensuelle de la consommation

- Q3 - Comparaison semaine vs week-end

Tables : "comparaison_jours". Logique : lecture des deux groupes (weekday, weekend) et calcul optionnel d'un écart en pourcentage.

```

SELECT
  day_type,
  avg_consumption,
  CASE
    WHEN day_type = 'weekend' THEN
      ROUND(((avg_consumption -
        (SELECT avg_consumption FROM comparaison_jours WHERE day_type = 'weekday')) /
        (SELECT avg_consumption FROM comparaison_jours WHERE day_type = 'weekday')) * 100, 2)
    ELSE NULL
  END AS difference_pourcent

FROM comparaison_jours
ORDER BY day_type;

```

Figure 28 : Résultats de la requête Q3 - Comparaison semaine / week-end

```
OK
weekday 1.0355 0.076 NULL
weekend 1.2342 0.078 NULL
Time taken: 30.395 seconds, Fetched: 2 row(s)
hive>
```

Figure 29 : Résultats de la requête Q3 - Comparaison semaine / week-end

- Q4 - Distribution horaire

Tables : "consumption_raw". Logique : agrégation par heure (SUBSTRING(time,1,2)) sur les mesures valides.

```
SELECT
SUBSTRING(`time`, 1, 2) AS heure,
COUNT(*) AS nombre_mesures,
AVG(global_active_power) AS consommation_moyenne
FROM consumption_raw
WHERE global_active_power IS NOT NULL
GROUP BY SUBSTRING(`time`, 1, 2)
ORDER BY consommation_moyenne DESC
LIMIT 10;
```

Figure 30 : Requête Hive Q4 - Distribution horaire de la consommation

```
OK
20      85615      1.8990641359574862
21      85529      1.8776974593412574
19      85542      1.7333350634775366
07      85284      1.5022463299094804
08      85295      1.4610164956914204
22      85553      1.412614730050403
09      85316      1.3316453420226007
18      85347      1.326451099628592
10      85248      1.260627439939952
11      85248      1.2458220016891985
Time taken: 29.892 seconds, Fetched: 10 row(s)
hive>
```

Figure 31 : Résultats de la requête Q4 - Distribution horaire de la consommation

- Q5 - Pic annuel

Tables : "consumption_raw". Logique : extraction de l'année depuis "date" via "YEAR(FROM_UNIXTIME(UNIX_TIMESTAMP(date, 'dd/MM/yyyy')))", puis "MAX(global_active_power)" (pic annuel) et "AVG(global_active_power)" (consommation moyenne annuelle). Cette requête s'appuie sur "consumption_raw" pour éviter toute ambiguïté liée au format des sorties agrégées (le commentaire dans "04_requetes_analytiques.hql" ligne 98-99 mentionne que "max_consumption" de "conso_par_jour" contient en fait le "count", d'où l'utilisation directe de "consumption_raw").

```
SELECT
CAST(YEAR(FROM_UNIXTIME(UNIX_TIMESTAMP(`date`, 'dd/MM/yyyy')))) AS STRING) AS annee,
MAX(global_active_power) AS pic_annuel,
AVG(global_active_power) AS consommation_moyenne
FROM consumption_raw
WHERE `date` IS NOT NULL
AND UNIX_TIMESTAMP(`date`, 'dd/MM/yyyy') IS NOT NULL
AND global_active_power IS NOT NULL
```

```
GROUP BY CAST(YEAR(FROM_UNIXTIME(UNIX_TIMESTAMP(`date`, 'dd/MM/yyyy')))) AS STRING)
ORDER BY annee;
```

Figure 32 : Requête Hive Q5 - Pic annuel de consommation

```
OK
2006      9.132      1.9012951073117563
2007     10.67      1.1170075009249016
2008     10.348      1.072096666381862
2009     11.122      1.078637301465425
2010      9.724      1.0610001486682799
Time taken: 33.144 seconds, Fetched: 5 row(s)
hive>
```

Figure 33 : Résultats de la requête Q5 - Pic annuel de consommation

7.4 Interprétation synthétique des résultats

- **Q1** : Met en évidence les journées extrêmes (top consommation moyenne), utiles pour cibler des dates à analyser plus finement (ex. 28/9/2010 avec 1.639 kW de moyenne).
 - **Q2** : Fournit une série mensuelle pour comparer les périodes dans le temps (variabilité et tendances), en sécurisant le parsing du format "dd/MM/yyyy".
 - **Q3** : Montre une différence entre les groupes "weekday" et "weekend" ; sur les sorties du projet, le week-end est plus élevé en moyenne (1.234 kW) que la semaine (1.036 kW).
 - **Q4** : Identifie les heures les plus consommatrices en moyenne ; dans les exports, le début de soirée ressort (20h avec 1.899 kW en moyenne, suivi de 21h avec 1.878 kW).
 - **Q5** : Compare les pics annuels ; les maxima observés sont de l'ordre de 9-11 kW, avec un maximum en 2009 (11.122 kW) sur l'export courant.
-

8. Exports & visualisations

8.1 Exports Hive vers HDFS

Les résultats Hive sont exportés en HDFS via le script "hive/scripts/05_exports.hql", en utilisant "INSERT OVERWRITE DIRECTORY".

Chaque export est écrit dans un répertoire dédié sous "/user/projet/export/" (un par analyse) : "q1_top_days", "q2_monthly_evolution", "q3_weekend_comparison", "q4_hourly_distribution", "q5_annual_peak".

Le format généré est un fichier texte (type CSV) ; les valeurs "NULL" sont représentées par "\N" dans les exports (observé). Les exports Q1 et Q3 contiennent un TAB entre la première colonne et les valeurs suivantes, hérité du format MapReduce source ("cle<TAB>valeurs_csv"), malgré le séparateur virgule défini dans le script d'export.

Preuve (exports présents dans HDFS) :

```
$ hdfs dfs -ls -R /user/projet/export | head
```

```
maxiimax@Max-Laptop-Yoga-Pro-I9:~/Bureau/FAC/M1/SEMESTRE-1/Cadre logiciel pour le big data/Chapitre-7/PROJET/script
on$ hdfs dfs -ls -R /user/projet/export | head
drwxr-xr-x  - maxiimax supergroup          0 2026-01-01 20:31 /user/projet/export/q1_top_days
-rw-r--r--  1 maxiimax supergroup        383 2026-01-01 20:30 /user/projet/export/q1_top_days/000000_0
drwxr-xr-x  - maxiimax supergroup          0 2026-01-01 20:31 /user/projet/export/q2_monthly_evolution
-rw-r--r--  1 maxiimax supergroup       2034 2026-01-01 20:31 /user/projet/export/q2_monthly_evolution/000000_0
drwxr-xr-x  - maxiimax supergroup          0 2026-01-01 20:31 /user/projet/export/q3_weekend_comparison
-rw-r--r--  1 maxiimax supergroup         80 2026-01-01 20:31 /user/projet/export/q3_weekend_comparison/000000_0
drwxr-xr-x  - maxiimax supergroup          0 2026-01-01 20:32 /user/projet/export/q4_hourly_distribution
-rw-r--r--  1 maxiimax supergroup        814 2026-01-01 20:32 /user/projet/export/q4_hourly_distribution/000000_0
drwxr-xr-x  - maxiimax supergroup          0 2026-01-01 20:33 /user/projet/export/q5_annual_peak
-rw-r--r--  1 maxiimax supergroup        150 2026-01-01 20:33 /user/projet/export/q5_annual_peak/000000_0
```

Figure 34 : Structure HDFS des exports Hive (commande `hdfs dfs -ls -R`)

8.2 Rapatriement local des résultats

Le rapatriement est automatisé via "scripts/export/export_results.sh". Le script utilise "hdfs dfs -getmerge" pour fusionner les fichiers d'un export HDFS en un fichier CSV local unique.

Organisation locale : les fichiers sont générés dans "data/export/" (un CSV par analyse).

Preuve (fichiers CSV présents) :

```
$ ls
```

```
maxiimax@Max-Laptop-Yoga-Pro-I9:~/Bureau/FAC/M1/SEMESTRE-1/Cadre logiciel pour le big data/Chapitre-7/PROJET/data/export$ ls
Q1_Top10_Jours.csv      Q3_Comparaison_Weekend.csv  Q5_Pic_Annuel.csv
Q2_Evolution_Mensuelle.csv  Q4_Distribution_Horaire.csv  README.md
```

Figure 35 : Fichiers CSV rapatriés localement (commande `ls data/export`)

8.3 Génération des visualisations

Les visualisations sont générées en local via "scripts/visualization/generate_graphs.py" (Python), à partir des CSV de "data/export/".

Bibliothèques principales : "pandas", "matplotlib", "seaborn". Le script gère le marqueur "\N" (NULL Hive) et utilise le séparateur TAB pour Q1/Q3, puis parse les valeurs séparées par virgule dans la deuxième colonne.

Fichiers PNG produits dans "data/visualizations/" :

- Q1_Top10_Jours.png
- Q2_Evolution_Mensuelle.png
- Q3_Comparaison_Weekend.png
- Q4_Distribution_Horaire.png
- Q5_Pic_Annuel.png

Preuve (fichiers présents) :

```
$ ls
```

```
maxiimax@Max-Laptop-Yoga-Pro-I9:~/Bureau/FAC/M1/SEMESTRE-1/Cadre logiciel pour le big data/Chapitre-7/PROJET/data/visualizations
$ ls
Q1_Top10_Jours.png  Q2_Evolution_Mensuelle.png  Q3_Comparaison_Weekend.png  Q4_Distribution_Horaire.png  Q5_Pic_Annuel.png
```

Figure 36 : Fichiers PNG de visualisation générés (commande ls data/visualizations)

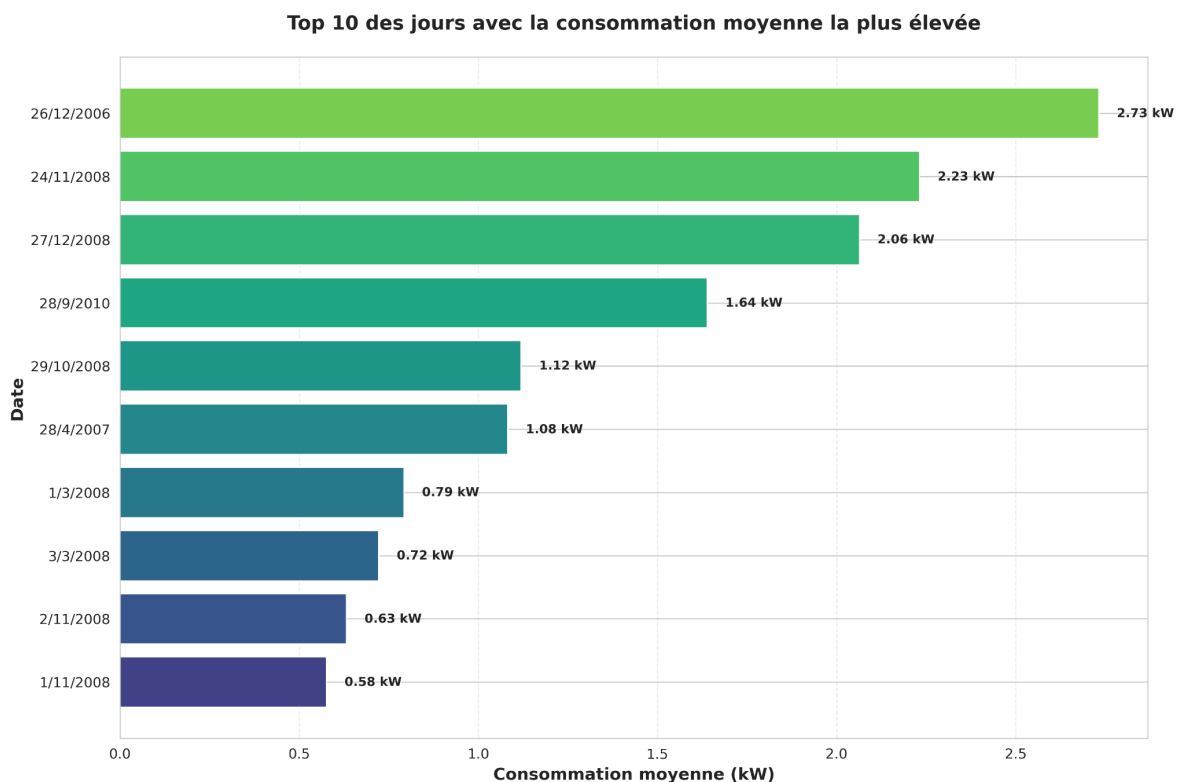


Figure 37 : Visualisation Q1 - Top 10 des jours avec la consommation moyenne la plus élevée

Évolution mensuelle de la consommation électrique

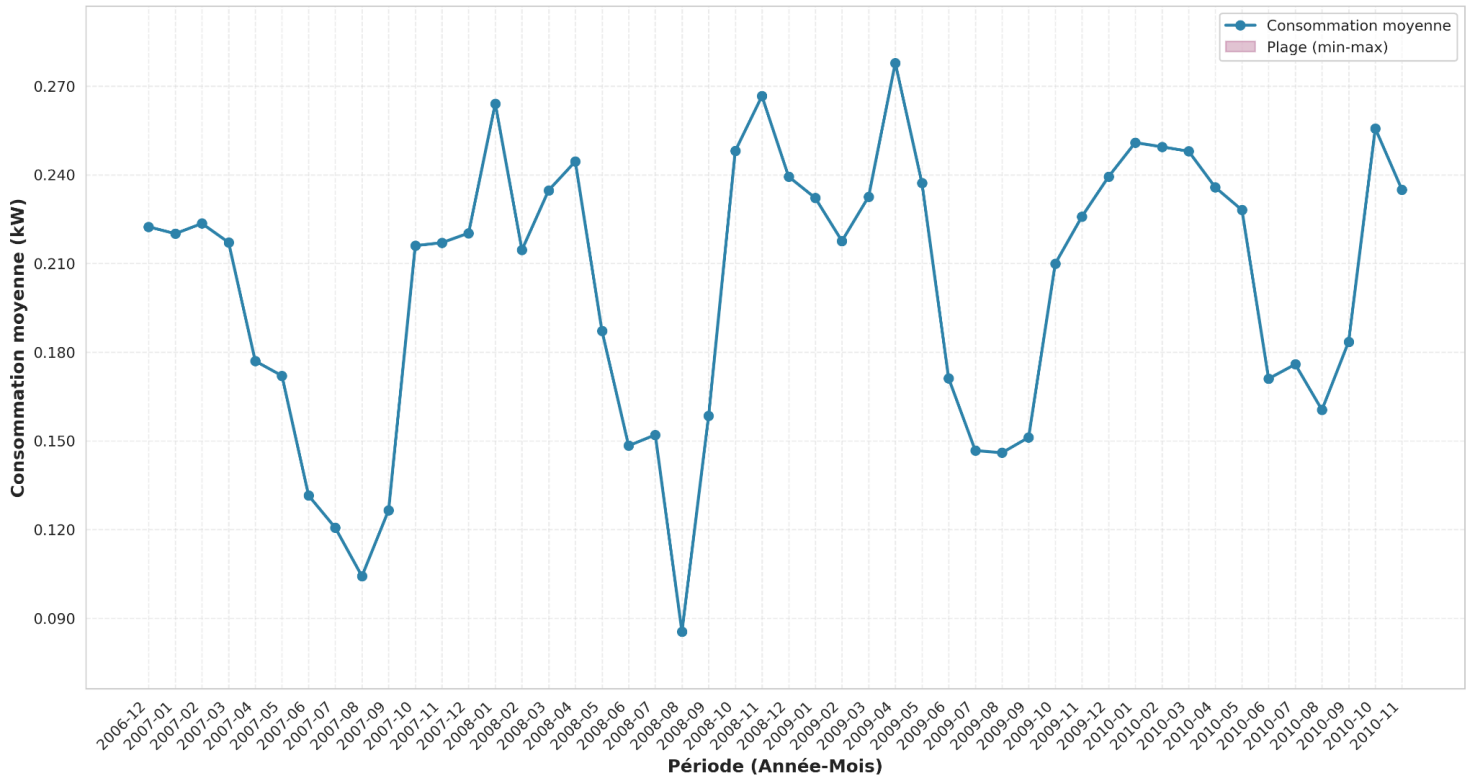


Figure 38 : Visualisation Q2 - Évolution mensuelle de la consommation électrique

Comparaison de consommation : Semaine vs Week-end

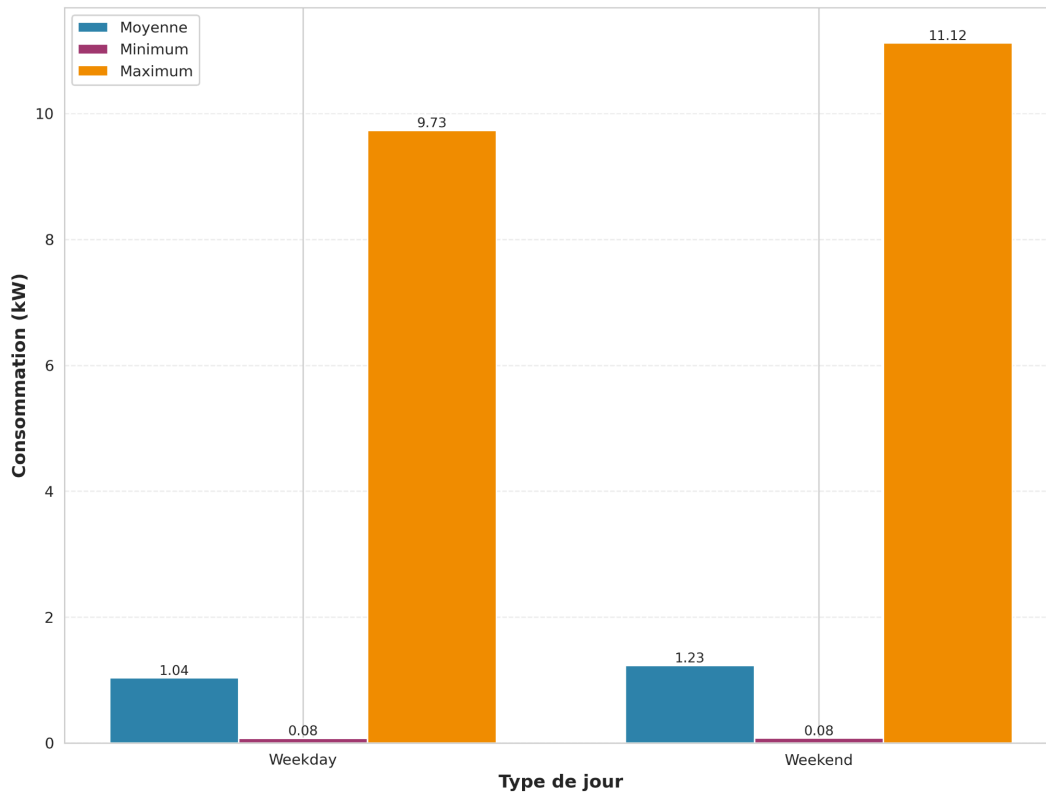


Figure 39 : Visualisation Q3 - Comparaison de la consommation électrique entre semaine et week-end

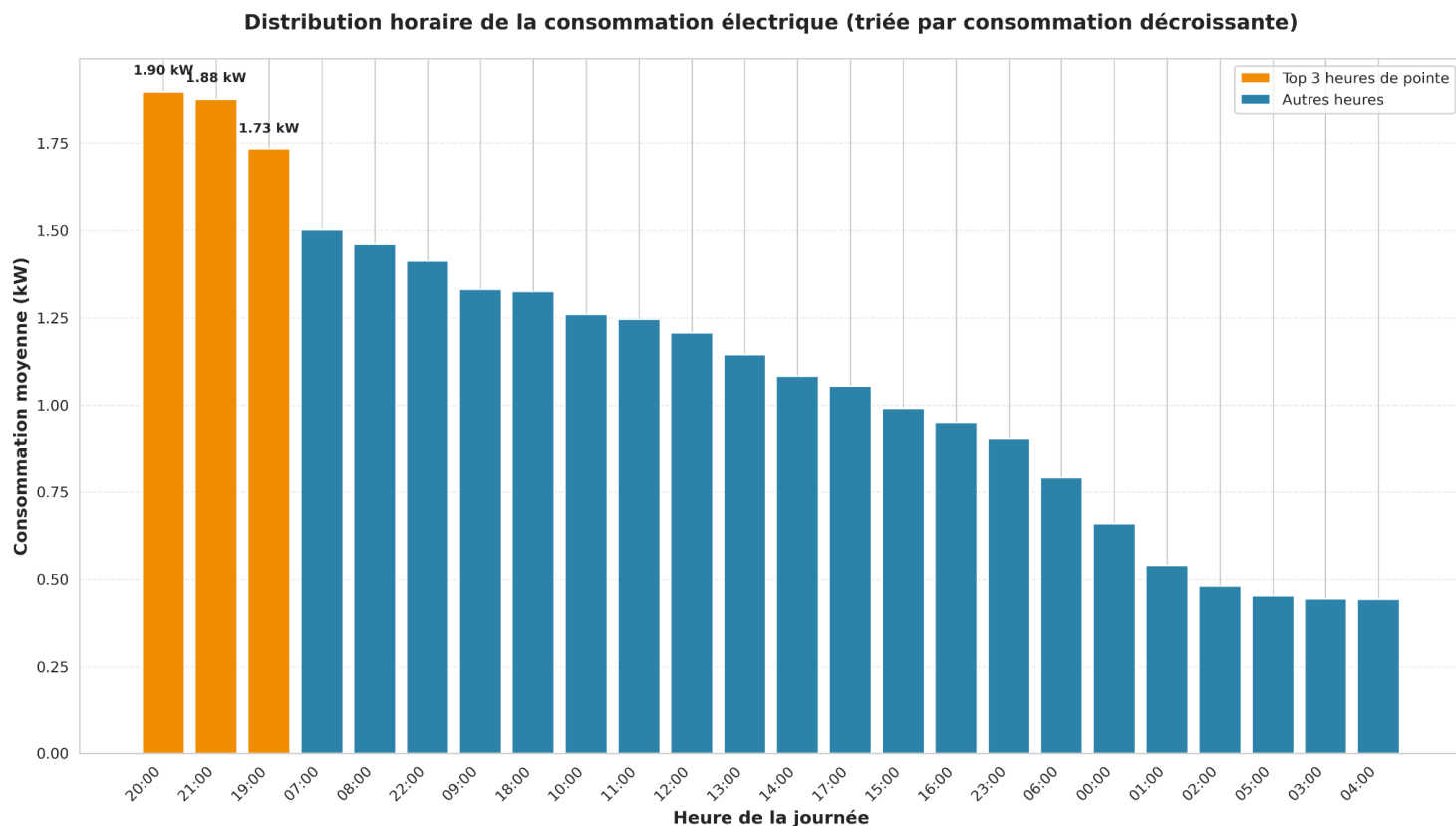


Figure 40 : Visualisation Q4 - Distribution horaire de la consommation électrique

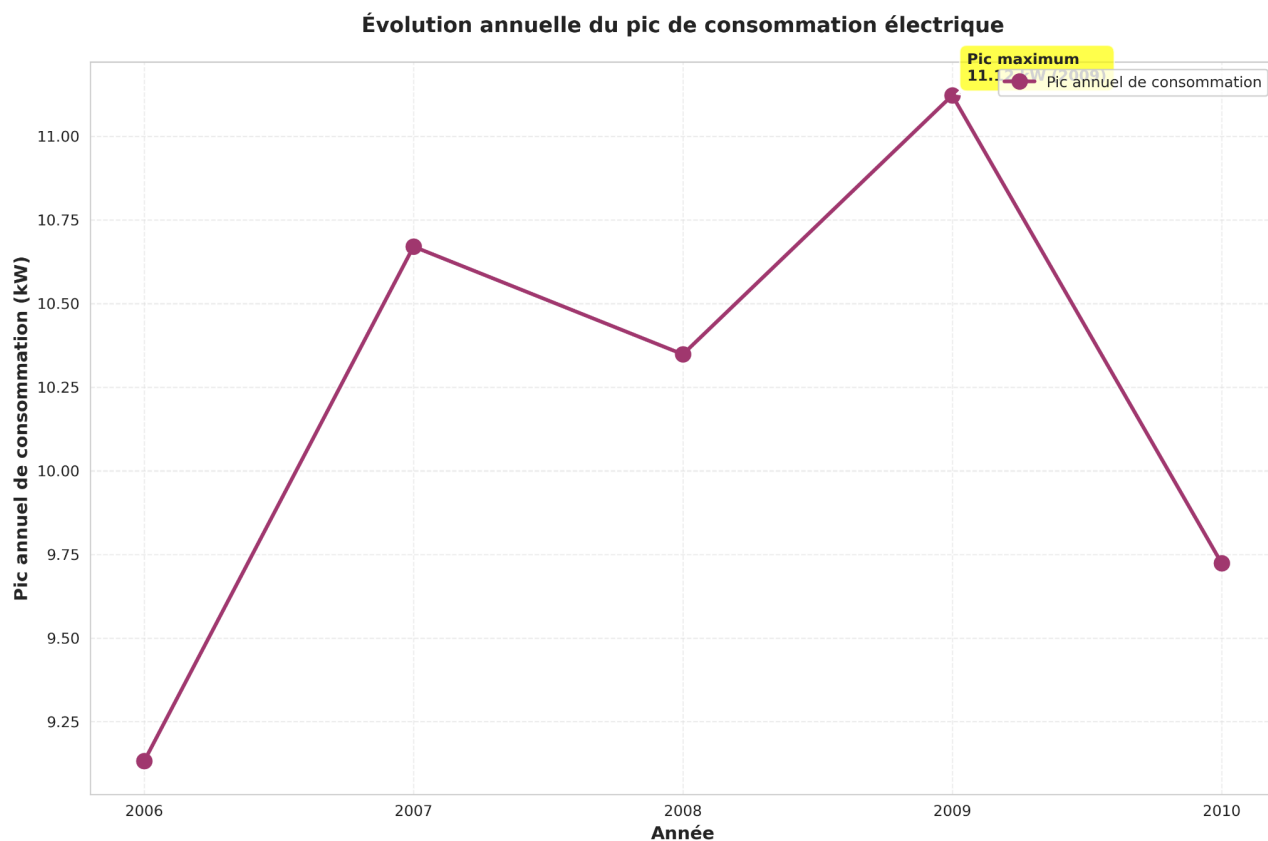


Figure 41 : Visualisation Q5 - Évolution annuelle du pic de consommation électrique

8.4 Analyse des résultats et interprétation des visualisations

Cette section présente l'interprétation des graphiques générés et les enseignements tirés de l'analyse des données de consommation électrique.

8.4.1 Analyse des pics et journées atypiques (Q1)

Identification des journées à forte consommation :

Le graphique met en évidence les 10 journées présentant la consommation électrique moyenne la plus élevée sur l'ensemble du jeu de données. La journée du 26/12/2006 ressort nettement comme un outlier majeur avec une consommation moyenne d'environ 2,73 kW, très supérieure aux autres valeurs. Deux autres journées se distinguent également : le 24/11/2008 (~2,23 kW) et le 27/12/2008 (~2,06 kW). Ces trois dates forment un premier groupe de journées à consommation exceptionnellement élevée, tandis que les autres journées du top 10 affichent des valeurs plus modérées, comprises entre 0,58 kW et 1,64 kW.

Patterns observés (dates, saisons) :

Un pattern saisonnier clair apparaît : la majorité des journées du top 10 se situe entre novembre et décembre, c'est-à-dire en période hivernale. Cela suggère une corrélation forte entre la consommation électrique et les besoins énergétiques liés au chauffage, à l'éclairage et à une occupation plus importante du logement. Une exception notable est le 28/09/2010, qui présente une consommation moyenne élevée (~1,64 kW) hors période hivernale, ce qui en fait une journée atypique du point de vue saisonnier. Les dates de mars et avril (2007–2008) apparaissent en bas du classement, avec des valeurs significativement plus faibles, confirmant que les pics majeurs sont majoritairement concentrés en hiver.

Interprétation des valeurs extrêmes :

Les valeurs les plus élevées, en particulier celle du 26/12/2006, peuvent s'expliquer par un cumul de facteurs, période post-Noël, forte présence au domicile, usage intensif des appareils électriques et conditions climatiques potentiellement rigoureuses. L'écart important entre cette journée et le reste du top 10 indique un comportement de consommation exceptionnel, qui peut être considéré comme une anomalie relative à la distribution globale. À l'inverse, les journées situées en bas du classement (autour de 0,6–0,8 kW) restent élevées par rapport à une consommation moyenne classique, mais ne relèvent pas de pics extrêmes. Ces résultats montrent l'intérêt d'identifier ces journées atypiques afin de mieux comprendre les facteurs contextuels influençant fortement la consommation et d'alimenter des analyses plus fines, notamment pour la détection d'anomalies ou la modélisation prédictive.

8.4.2 Évolution temporelle et saisonnalité (Q2, Q5)

Tendance générale sur la période 2006–2010 :

Sur la période 2006–2010, l'évolution mensuelle de la consommation moyenne ne met pas en évidence de tendance globale marquée (ni croissance, ni décroissance continue). Les valeurs restent globalement stationnaires, avec des fluctuations régulières d'un mois à l'autre et d'une année sur l'autre. Cette stabilité relative suggère que la série est principalement gouvernée par des facteurs saisonniers et contextuels plutôt que par une évolution structurelle de long terme. Le graphique des pics annuels confirme cette observation, bien que les maxima varient selon les années, ils restent dans un intervalle borné, indiquant l'absence de changement brutal du régime de consommation.

Saisonnalité (été / hiver, mois de forte et faible consommation) :

La saisonnalité est clairement identifiable dans la série temporelle mensuelle. Les niveaux de consommation les plus élevés se situent majoritairement en période hivernale (fin d'année et début d'année), tandis que les minima sont observés de façon récurrente durant l'été, notamment entre juillet et septembre. Ce comportement cyclique est cohérent avec une augmentation des usages énergétiques liés au chauffage et à l'éclairage en hiver, et une diminution des besoins énergétiques en été. La répétition de ce schéma sur plusieurs années indique une saisonnalité stable, ce qui est un point clé pour toute modélisation prédictive ou approche de décomposition de séries temporelles.

Évolution des pics annuels :

L'analyse des pics annuels montre une forte hausse entre 2006 et 2007, suivie de variations plus modérées jusqu'en 2010. Le pic maximal est atteint en 2009, avec une valeur supérieure à 11 kW, ce qui correspond à un événement extrême sur l'ensemble de la période étudiée. La baisse observée en 2010 suggère que ces pics ne suivent pas une tendance monotone, mais sont fortement dépendants de conditions ponctuelles (climat exceptionnel, usages spécifiques). D'un point de vue Big Data, ces valeurs extrêmes constituent des points d'intérêt pour des analyses de détection d'anomalies et pour l'évaluation des charges maximales dans une logique de dimensionnement ou de prévision de la demande.

8.4.3 Patterns journaliers et horaires (Q3, Q4)

Différence de consommation entre semaine et week-end :

La comparaison entre les jours de semaine et le week-end met en évidence une différence significative de niveau de consommation. La consommation moyenne est plus élevée le week-end, avec un écart d'environ 19 % par rapport aux jours ouvrés. Cette hausse s'explique par une présence plus importante au domicile et une utilisation accrue des équipements électriques (cuisine, loisirs, multimédia). En revanche, les valeurs minimales restent similaires entre semaine et week-end, ce qui suggère un socle de consommation incompressible relativement constant, indépendamment du type de jour. Les valeurs maximales, plus élevées le week-end, traduisent une plus grande probabilité de pics liés à des usages simultanés.

Heures de pointe (20h-21h) :

L'analyse de la distribution horaire, triée par consommation décroissante, montre clairement que les heures de pointe se situent en début de soirée, avec un maximum entre 20h et 21h, suivi de près par 19h. Ces créneaux correspondent aux niveaux de consommation moyens les plus élevés de la journée. À l'inverse, les heures nocturnes (entre 1h et 5h du matin) présentent les consommations les plus faibles, ce qui confirme une forte variabilité intra-journalière. Cette structure horaire est typique d'un usage résidentiel, avec une montée progressive de la consommation en fin de journée.

Interprétation des comportements domestiques :

Les patterns observés traduisent des comportements domestiques cohérents avec un cadre résidentiel classique. Les pics en soirée correspondent à la convergence de plusieurs usages énergivores, préparation des repas, éclairage, chauffage, et activités de loisir après la journée de travail. La surconsommation observée le week-end renforce cette interprétation, en indiquant une intensification et un étalement temporel des usages. D'un point de vue Big Data, ces résultats sont particulièrement pertinents pour la segmentation temporelle de la consommation, la modélisation des profils d'usagers et l'optimisation de la gestion de la demande, notamment dans des contextes de prévision horaire ou de pilotage énergétique.

8.4.4 Synthèse et enseignements

Principaux patterns identifiés :

L'ensemble des analyses met en évidence plusieurs patterns structurants de la consommation électrique. Tout d'abord, une saisonnalité marquée se dégage, avec des niveaux de consommation plus élevés en période hivernale et des creux récurrents durant l'été. À une échelle plus fine, des patterns journaliers et horaires apparaissent clairement, notamment une surconsommation le week-end par rapport aux jours ouvrés, ainsi que des heures de pointe concentrées en début de soirée (autour de 19h–21h). Enfin, l'identification de journées et de pics annuels atypiques souligne l'existence de comportements extrêmes, souvent liés à des conditions ponctuelles ou exceptionnelles.

Cohérence entre les différentes analyses :

Les résultats obtenus aux différentes granularités temporelles (horaire, journalière, mensuelle et annuelle) sont globalement cohérents entre eux. Les pics journaliers et annuels observés s'inscrivent logiquement dans les périodes de forte consommation identifiées à l'échelle mensuelle, principalement en hiver. De même, les heures de pointe horaires contribuent directement aux maxima journaliers et annuels. Cette cohérence inter-niveaux renforce la robustesse des observations et suggère que les patterns mis en évidence traduisent des comportements réels et stables, plutôt que des artefacts liés au traitement des données.

Limites de l'interprétation (données d'un seul foyer) :

Malgré la richesse temporelle du jeu de données, l'interprétation des résultats reste contrainte par le fait qu'ils proviennent d'un seul foyer. Les patterns identifiés ne peuvent donc pas être généralisés sans précaution à une population plus large, en raison de la forte dépendance aux habitudes de vie, au type de logement et aux équipements spécifiques. Par ailleurs, l'absence de variables contextuelles explicites (conditions météorologiques précises, nombre d'occupants, événements exceptionnels) limite la capacité à expliquer finement certaines variations ou pics extrêmes. Ces limites ouvrent néanmoins des perspectives intéressantes pour des travaux futurs, notamment l'agrégation de données multi-foyers ou l'intégration de données exogènes dans une approche Big Data plus globale.

9. Tests, validations & qualité

9.1 Tests réalisés

Les vérifications suivantes ont été effectuées sur la chaîne complète (HDFS → MapReduce → exports Hive → rapatriement local → visualisations) :

Le workflow du projet s'appuie sur une chaîne de scripts automatisés couvrant l'ensemble du traitement des données. La structure des répertoires HDFS est d'abord créée et vérifiée à l'aide du script "scripts/hdfs/create_structure.sh". Les traitements distribués sont ensuite exécutés via le lancement des trois jobs MapReduce avec "scripts/execution/run_mapreduce_jobs.sh", en contrôlant systématiquement la présence des sorties dans HDFS. Une fois les calculs terminés, les résultats sont exportés depuis Hive par la création des répertoires "/user/projet/export/..." à l'aide du script "hive/scripts/05_exports.hql". Enfin, les données sont rapatriées localement sous forme de fichiers CSV grâce à "scripts/export/export_results.sh", puis exploitées pour la génération des visualisations graphiques au format PNG via "scripts/visualization/generate_graphs.py".

9.2 Validations fonctionnelles

- **Sorties MapReduce** : Présence des fichiers "part-r-*" et du marqueur "_SUCCESS" dans chaque répertoire de job (ex. "job1_region_avg").

Par la suite, cinq fichiers CSV ont été créés, ainsi que 5 visualisations graphiques générées à partir des données traitées avec Python.

```
hdfs dfs -ls /user/projet/output/job1_region_avg
ls -lh data/export data/visualizations
```

```
maxiimax@Max-Laptop-Yoga-Pro-19:~/Bureau/FAC/M1/SEMESTRE-1/Cadre logiciel pour le big data/Chapitre-7/PROJET$ ls -lh data/export
data/visualizations
data/export:
total 32K
-rw-r--r-- 1 maxiimax maxiimax 383 janv. 1 20:33 Q1_Top10_Jours.csv
-rw-r--r-- 1 maxiimax maxiimax 2,0K janv. 1 20:33 Q2_Evolution_Mensuelle.csv
-rw-r--r-- 1 maxiimax maxiimax 80 janv. 1 20:33 Q3_Comparaison_Weekend.csv
-rw-r--r-- 1 maxiimax maxiimax 814 janv. 1 20:33 Q4_Distribution_Horaire.csv
-rw-r--r-- 1 maxiimax maxiimax 150 janv. 1 20:33 Q5_Pic_Annuel.csv
-rw-rw-r-- 1 maxiimax maxiimax 9,8K déc. 27 18:29 README.md

data/visualizations:
total 1,3M
-rw-rw-r-- 1 maxiimax maxiimax 204K janv. 1 20:33 Q1_Top10_Jours.png
-rw-rw-r-- 1 maxiimax maxiimax 437K janv. 1 20:33 Q2_Evolution_Mensuelle.png
-rw-rw-r-- 1 maxiimax maxiimax 126K janv. 1 20:33 Q3_Comparaison_Weekend.png
-rw-rw-r-- 1 maxiimax maxiimax 214K janv. 1 20:33 Q4_Distribution_Horaire.png
-rw-rw-r-- 1 maxiimax maxiimax 252K janv. 1 20:33 Q5_Pic_Annuel.png
```

Figure 42 : Rapatriement local et génération des graphiques - Fichiers CSV et PNG

9.3 Indicateurs de qualité et limites

- **Complétude** : Tous les artefacts attendus sont présents (3 outputs MapReduce, 5 répertoires d'export HDFS, 5 CSV locaux, 5 PNG).
- **Cohérence de format** : Dates au format "dd/MM/yyyy" et gestion des "NULL" Hive via "\N". Les champs "\N" observés en fin de ligne dans les exports Q1 et Q3 proviennent du problème d'alignement format MapReduce/Hive documenté en section 7.2 (les requêtes fonctionnent néanmoins correctement).
- **Limites** : Les métriques d'exécution (temps, ressources YARN) ne sont pas documentées précisément dans ce rapport, elles peuvent être consultées via l'UI YARN si nécessaire.

10. Conclusion, limites, perspectives

11.1 Bilan du projet

Ce projet met en place une chaîne Hadoop complète HDFS → MapReduce sous YARN → Hive → exports → visualisations appliquée à un dataset de consommation électrique.

Les objectifs principaux sont atteints : ingestion maîtrisée dans HDFS, exécution de 3 jobs MapReduce, modélisation Hive via tables externes, requêtes analytiques Q1-Q5, exports HDFS puis rapatriement local et génération des graphes.

Le livrable final combine scripts d'exécution, scripts Hive et éléments de preuve (présence des répertoires HDFS, outputs, exports CSV et PNG).

Cette approche fournit un pipeline reproductible et cohérent avec les exigences du cours, sans prétention d'optimisation avancée.

11.2 Limites identifiées

- **Mode pseudo-distribué** : Exécution sur un seul nœud, donc pas de validation de scalabilité ni de tolérance aux pannes multi-nœuds.
- **Formats de stockage** : Tables en "TEXTFILE" (pas de format colonne type ORC/Parquet ni compression au niveau table Hive).
- **Mesures de performance** : Temps d'exécution et ressources YARN non documentés précisément dans ce rapport, elles peuvent être consultées via l'UI YARN (section 4.6) si nécessaire.
- **Formats et parsing** : Dépendance au format de date "dd/MM/yyyy" (parsing via "UNIX_TIMESTAMP") et problème d'alignement "TAB + CSV" des outputs MapReduce → Hive (documenté en section 7.2).
- **Volumétrie** : Dataset de 127 MB, représentatif d'un traitement batch mais limité pour évaluer des gains de performance à grande échelle.

11.3 Perspectives d'amélioration

1. **Optimisation Hive (stockage et requêtes)** : Convertir les tables en ORC/Parquet et partitionner par période (ex. année/mois) afin de réduire l'I/O et accélérer les agrégations temporelles (Hive 3.1.3 supporte ces formats).
 2. **Robustesse des formats MapReduce ↔ Hive** : Standardiser le format de sortie (délimiteur unique, schéma stable) pour supprimer les ambiguïtés "TAB + CSV" documentées en section 7.2 et fiabiliser les exports.
 3. **Optimisation MapReduce** : Introduire un combiner lorsque applicable (réduction du volume de données shuffle) et activer la compression des sorties intermédiaires (codec Snappy) pour réduire le transfert réseau.
 4. **Orchestration et reproductibilité** : Regrouper les étapes (ingestion, jobs, Hive, exports, graphes) dans un script "pipeline" unique avec gestion d'erreurs, nettoyage des outputs et reprise sur erreur.
 5. **Montée en charge** : Exécuter sur un cluster multi-nœuds et/ou sur un volume plus important pour valider la distribution réelle et documenter des métriques d'exécution via YARN.
-

11. Bibliographie

1. Documentation technique officielle

Apache Hadoop

Apache Software Foundation. (2023). *Apache Hadoop*.
<https://hadoop.apache.org/>

Description : Source officielle présentant l'architecture globale de Hadoop, ses composants principaux (HDFS, YARN, MapReduce) et ses cas d'usage en Big Data.

Apache Software Foundation. (2023). *Hadoop Documentation – HDFS User Guide*.
<https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsUserGuide.html>

Description : Utilisé pour décrire le fonctionnement du système de fichiers distribué HDFS (stockage, réplication, tolérance aux pannes).

Apache Software Foundation. (2023). *Hadoop Documentation – YARN ResourceManager*.
<https://hadoop.apache.org/docs/stable/hadoop-yarn/hadoop-yarn-site/YARN.html>

Description : Référence pour expliquer la gestion des ressources et l'ordonnancement des applications distribuées via YARN.

Apache Software Foundation. (2023). *Hadoop MapReduce Tutorial*.
<https://hadoop.apache.org/docs/stable/hadoop/mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>

Description : Utilisé pour introduire le modèle de programmation MapReduce et le principe du traitement distribué par lots.

Apache Software Foundation. (2023). *Hadoop 3.3.6 Release Notes*.
<https://hadoop.apache.org/docs/r3.3.6/>

Description : Source utilisée pour justifier la version de Hadoop employée et mentionner les évolutions techniques et correctifs associés.

Apache Hive

Apache Software Foundation. (2023). *Apache Hive*.
<https://hive.apache.org/>

Description : Source officielle présentant Apache Hive, son rôle dans l'écosystème Hadoop et son usage pour l'analyse de données massives via un langage SQL-like (HiveQL) au-dessus de HDFS.

Apache Software Foundation. (2023). *Hive Language Manual*.
<https://cwiki.apache.org/confluence/display/Hive>

Description : Utilisé pour décrire la syntaxe HiveQL, les types de tables (internes, externes), les partitions, ainsi que les mécanismes d'exécution des requêtes Hive.

Apache Software Foundation. (2023). *Hive Data Warehouse Software – User Documentation.*

<https://hive.apache.org/userdoc/>

Description : Utilisé pour détailler l'utilisation opérationnelle d'Apache Hive comme entrepôt de données distribué, incluant la création de tables, la gestion des partitions, l'optimisation des requêtes et l'intégration avec HDFS et YARN.

2. Dataset et sources de données

Lichman, M. (2013). *UCI Machine Learning Repository: Individual household electric power consumption data set.* University of California, Irvine, School of Information and Computer Sciences.

<https://archive.ics.uci.edu/ml/datasets/individual+household+electric+power+consumption>

Description : Source officielle du dépôt UCI utilisée pour référencer le jeu de données de consommation électrique domestique, incluant sa description générale, ses variables et son contexte de collecte.

Hebrail, G., & Berard, A. (2012). *Individual household electric power consumption data set.* UCI Machine Learning Repository.

<https://archive.ics.uci.edu/dataset/235/individual+household+electric+power+consumption>

Description : Utilisé pour détailler l'origine du jeu de données, la méthodologie de mesure, la période d'observation (2006–2010) et les caractéristiques statistiques exploitées dans les analyses.

3. Guides et tutoriels techniques

Installation et configuration

YouTube. (n.d.). *Installation Hadoop* [Vidéo].

https://www.youtube.com/watch?v=eBIDBUy_610

Description : Utilisée comme support pédagogique pour illustrer les étapes pratiques d'installation et de configuration initiale de Hadoop dans un environnement local.

YouTube. (n.d.). *Configuration Hive* [Vidéo].

<https://www.youtube.com/watch?v=LMrW2BFh8>

Description : Utilisée pour accompagner la mise en place et la configuration d'Apache Hive, en complément de la documentation officielle, afin de faciliter la compréhension des manipulations techniques.

Références Java MapReduce

Oracle Corporation. (2023). *Java Platform, Standard Edition Documentation.*

<https://docs.oracle.com/javase/8/docs/>

Description : Utilisée pour référencer l'environnement Java Standard Edition, indispensable à l'exécution et à la configuration des composants de l'écosystème Hadoop, ainsi qu'à la compréhension des dépendances Java utilisées.

Maven Project. (2023). *Maven – Guide d'utilisation.*

<https://maven.apache.org/guides/>

Description : Utilisé pour documenter la gestion des dépendances, la structuration des projets Java et l'automatisation du processus de build dans les applications liées à Hadoop et Hive.